

ENTERPRISE TECHNICAL TRAINING

AI, Machine Learning, Deep Learning, Generative AI & Agentic AI with Oracle Cloud Infrastructure

A progressive technical curriculum for Oracle software professionals, senior engineers, developers, data engineers, ML engineers, cloud engineers, and solution architects.

COURSE ROADMAP

AI → ML → DL → Neural Networks → Transformers → GenAI → Agentic AI → OCI

14 modules progressing from AI foundations to production-grade Oracle Cloud Infrastructure architectures.

- | | | | |
|-----------|-------------------------------|-----------|--------------------------------|
| 01 | Artificial Intelligence | 08 | Transformer Architecture |
| 02 | Machine Learning | 09 | Generative AI & RAG |
| 03 | Deep Learning | 10 | Prompt Engineering |
| 04 | Neural Networks & ANN | 11 | Agentic AI |
| 05 | Convolutional Neural Networks | 12 | Oracle Cloud Infrastructure |
| 06 | Recurrent Neural Networks | 13 | OCI Enterprise AI Architecture |
| 07 | LSTM Networks | 14 | Oracle Enterprise Use Cases |

MODULE

01

Artificial Intelligence Foundations

Learning Objectives

- Define AI precisely and place it in the AI/ML/DL/GenAI/Agentic hierarchy
- Trace the evolution of AI from symbolic systems to agentic systems
- Differentiate Narrow, General, and Super AI, and rule-based vs learned systems
- Recognize AI patterns already present in enterprise and Oracle platforms

Definition of Artificial Intelligence

OBJECTIVE *Establish a precise, enterprise-usable definition of AI.*

- AI is the field of computer science focused on building systems that perform tasks normally requiring human intelligence: perception, reasoning, decision-making, language, and planning.

Formal framing (Russell & Norvig): an agent that acts rationally — selecting actions to maximize expected goal achievement given its percepts and knowledge.

- AI is an umbrella discipline. Machine Learning, Deep Learning, Generative AI, and Agentic AI are progressively specialized subsets — not synonyms for AI itself.
- Enterprise AI systems are judged on accuracy, latency, cost-per-inference, explainability, and governance — not novelty. This is the lens used throughout this course.

ENTERPRISE EXAMPLE

Oracle Fusion ERP embeds AI-driven transaction anomaly detection and intelligent document recognition without customers building models themselves.

KEY DISTINCTION

AI describes the capability. ML/DL describe how the capability is learned from data. GenAI describes what the system produces. Agentic AI describes how autonomously the system acts to reach a goal.

Evolution of AI

Six eras of AI capability



Six decades of AI moved from hand-coded logic, to statistical learning, to representation learning, to generative and now autonomous systems.

Types of AI by Capability

OBJECTIVE *Distinguish AI systems by the breadth of intelligence they exhibit.*

- **Narrow AI (ANI):**

Performs one task or a tightly bounded set of tasks extremely well (translation, fraud scoring, image classification). Every production AI system in the enterprise today — including every system in this course — is Narrow AI.

- **General AI (AGI):**

A hypothetical system with human-equivalent reasoning across arbitrary domains, able to transfer learning between unrelated tasks without retraining. Not yet achieved; an active research goal.

- **Super AI (ASI):**

A theoretical system exceeding human intelligence across all domains. Purely speculative — relevant to AI safety research, not to enterprise architecture.

PRACTICAL TAKEAWAY

When an OCI solution architect says "AI system," they mean a Narrow AI component solving one well-scoped business problem — never AGI or ASI. Scope every AI initiative accordingly.

Rule-Based AI vs. Learning-Based AI

OBJECTIVE *Contrast explicit-logic systems with data-driven systems.*

- Rule-based AI encodes human expertise as explicit if-then logic, decision tables, or business rules. Behavior is fully deterministic and fully explainable.

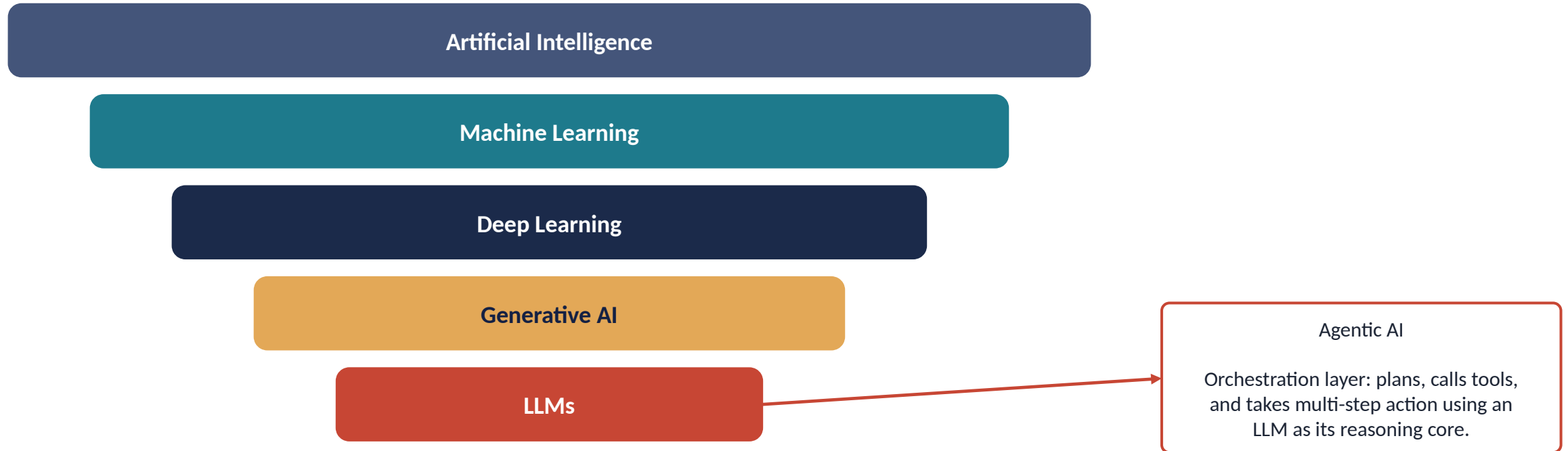
Example: a PL/SQL decision procedure that flags an invoice when amount > threshold AND vendor is new — deterministic, auditable, but brittle to unseen patterns.

- Learning-based AI (ML/DL) infers decision boundaries statistically from historical data. It generalizes to unseen cases but requires data, training, monitoring, and drift management.
- Most mature enterprises run a hybrid: rule-based guardrails wrap a learned model's output — e.g., an ML fraud score is always overridden by a hard compliance rule.
- This hybrid pattern reappears in Module 11 as a guardrail around Agentic AI actions.

ENTERPRISE EXAMPLE

Oracle Cloud EPM combines rule-based validation (chart-of-accounts mapping rules) with ML-based anomaly detection on close-process transactions.

AI Hierarchy — Nested Technology Layers



Each inner layer is a specialized subset of the layer surrounding it. Agentic AI is an orchestration layer built on top of Generative AI models.

Reading the AI Hierarchy

Each layer inherits the properties of the layer around it, then adds a specialization:

- 1 AI (outermost):** Any system exhibiting intelligent behavior, learned or rule-based.
- 2 Machine Learning:** AI systems that improve automatically by learning statistical patterns from data instead of being explicitly programmed.
- 3 Deep Learning:** ML using multi-layer neural networks to learn hierarchical feature representations directly from raw data.
- 4 Generative AI:** Deep Learning models (mostly Transformer-based) trained to generate new content — text, code, images — rather than only classify or predict.
- 5 Agentic AI (adjacent, not nested):** Uses an LLM as a reasoning engine inside a loop that plans, calls tools, and acts — it is built on GenAI, not merely a subset of it.

AI Examples in Enterprise Environments

OBJECTIVE *Recognize recurring enterprise AI patterns before mapping them to OCI in Module 12.*

- Fraud and anomaly detection — transaction scoring, expense-report outlier flagging.
- Forecasting — demand planning, revenue forecasting, capacity planning.
- Document intelligence — invoice extraction, contract clause classification, KYC document verification.
- Conversational interfaces — IT helpdesk bots, HR self-service assistants, customer support copilots.
- Predictive maintenance — equipment failure prediction from sensor telemetry.
- Personalization and recommendation — next-best-action for sales, targeted offers.
- Code and data assistance — SQL generation, code review, log-analysis copilots for engineering teams.

Oracle-Related AI Examples

OBJECTIVE *See where Oracle already ships AI inside its stack — the foundation for Module 12–14.*

- Oracle Fusion Applications — embedded ML for demand forecasting, expense anomaly detection, and intelligent document recognition in Payables.
- Oracle Autonomous Database — self-driving (automated tuning), self-securing, and self-repairing capabilities powered by internal ML.
- Oracle Digital Assistant — conversational AI/NLU for enterprise self-service, built on Oracle's own NLP engine.
- OCI AI Services — pretrained Vision, Language, Speech, and Document Understanding APIs consumable without training a model.
- OCI Generative AI — managed access to foundation LLMs (Cohere, Meta Llama family) for chat, embeddings, summarization, and RAG.
- OCI Generative AI Agents — managed retrieval-augmented agent service with SQL, RAG, and function-calling tools (covered in depth in Modules 12–13).

ENTERPRISE EXAMPLE

A single Oracle customer can combine Autonomous Database self-tuning (rule/heuristic-driven), Fusion ML forecasting, and an OCI Generative AI chatbot — three different points on the AI spectrum in one landscape.

AI vs ML vs DL vs GenAI vs Agentic AI

Dimension	AI	ML	DL	Generative AI	Agentic AI
Definition	Umbrella field for intelligent behavior	Learns patterns from data	ML using layered neural networks	Produces new content/data	Plans & acts autonomously using tools
Core technique	Rules, search, learning, logic	Regression, trees, SVM, clustering	CNN, RNN, LSTM, Transformer	Foundation models (LLMs, diffusion)	LLM + planning + tools + memory
Typical output	A decision or action	A prediction or label	A prediction or learned representation	Text, code, image, audio	A completed multi-step task
Data need	Varies (rules need none)	Structured, labeled data	Large labeled/unlabeled datasets	Massive pretraining corpora	An LLM + tools; little/no new training
Enterprise example	Rule-based approval workflow	Credit/fraud scoring model	Image-based defect classifier	Draft an incident report from logs	Diagnose, remediate, and report an incident end-to-end

Machine Learning

Learning Objectives

- Define ML and walk the full data-to-deployment workflow
- Differentiate supervised, unsupervised, and reinforcement learning
- Apply regression, classification, and clustering to the right problem type
- Evaluate models correctly and recognize overfitting vs. underfitting

Definition of Machine Learning

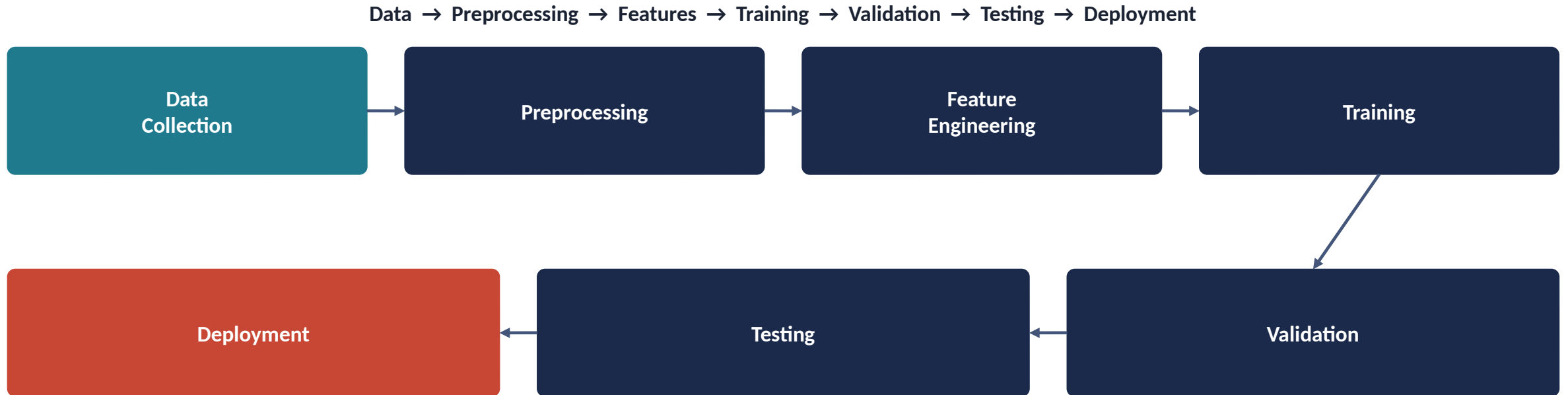
OBJECTIVE *Define ML precisely as a data-driven alternative to explicit programming.*

- Machine Learning is a subset of AI in which a system improves its performance on a task by learning statistical patterns from data, rather than following explicitly programmed rules (Tom Mitchell's formal definition: improving performance measure P on task T through experience E).
- The output of ML is a trained model: a mathematical function whose parameters were fit to historical data and that now generalizes to new, unseen inputs.
- ML shifts engineering effort from writing logic to curating data, engineering features, and validating model behavior.
- Every ML system carries model risk: performance degrades as real-world data drifts away from the training distribution — this motivates the validation, monitoring, and governance steps covered later in this module.

FORMAL VIEW

A model is a function $f(x; \theta)$ with parameters θ learned by minimizing a loss function over training data. Module 3 extends this into neural networks and gradient-based training.

The Machine Learning Lifecycle



Data flows through seven stages from raw source to production inference; validation and testing gate promotion to deployment.

Walking the ML Lifecycle

- 1 Data collection:** Gather representative historical data covering the range of inputs the model will see in production.
- 2 Preprocessing:** Clean nulls/outliers, normalize scales, encode categoricals, and split into train / validation / test sets.
- 3 Feature engineering:** Transform raw fields into predictive signals (ratios, aggregates, embeddings) — often the highest-leverage step.
- 4 Training:** Fit model parameters by minimizing a loss function against the training set.
- 5 Validation:** Tune hyperparameters on a held-out validation set to avoid overfitting to training data.
- 6 Testing:** Report unbiased performance on a test set the model has never influenced.
- 7 Deployment:** Serve the model behind an API, then monitor for data/concept drift and retrain on a schedule.

Supervised Learning

OBJECTIVE *Learn a mapping from inputs to known, labeled outputs.*

- Training data consists of (input, correct-output) pairs; the model learns to predict the output for new inputs.
- Two families: regression (continuous output) and classification (discrete/categorical output) — detailed on the next slide.
- Requires labeled data, which is often the most expensive part of a supervised project — label quality bounds model quality.
- Dominant algorithms: linear/logistic regression, decision trees, random forests, gradient boosting (XGBoost/LightGBM), support vector machines.

ENTERPRISE EXAMPLE

Predicting whether an Oracle ERP invoice will be paid late, trained on years of historical AP transactions labeled with actual payment outcome.

Unsupervised Learning

OBJECTIVE *Discover structure in data with no labeled outcome.*

- Training data has inputs only — no correct-answer labels. The model finds structure: groupings, densities, or lower-dimensional representations.
- Core tasks: clustering (group similar records), dimensionality reduction (PCA, t-SNE, UMAP), and association-rule mining (market-basket analysis).
- Used heavily for exploratory analysis and as a preprocessing step before supervised modeling (e.g., cluster-derived features).
- Embeddings — dense vector representations learned in an unsupervised or self-supervised way — are the unsupervised technique most relevant to this course; they underpin semantic search and RAG (Module 9).

ENTERPRISE EXAMPLE

Segmenting manufacturing customers into behavioral clusters from order-history data to target account-management effort.

Reinforcement Learning

OBJECTIVE *Learn a policy by acting in an environment and receiving reward feedback.*

- An agent takes actions in an environment; each action returns a reward signal and a new state. The agent learns a policy that maximizes cumulative reward over time — no labeled dataset is required.
- Core components: agent, environment, state, action, reward, policy — the same vocabulary reappears in Module 11 (Agentic AI), which borrows RL's action/observation loop conceptually.
- Distinct from supervised learning: feedback is delayed and evaluative (a reward score), not an immediate correct label.
- In modern LLM training, Reinforcement Learning from Human Feedback (RLHF) fine-tunes a base model's outputs to align with human preferences — the technique behind why chat-tuned models follow instructions.

ENTERPRISE EXAMPLE

OCI data-center cooling and resource-scheduling optimization problems are natural RL formulations: actions affect state (temperature, load) and are scored by an efficiency reward.

Regression vs. Classification vs. Clustering

Task	Learning Type	Output	Typical Algorithms	Enterprise Example
Regression	Supervised	Continuous number	Linear regression, gradient boosting	Forecast next quarter's revenue
Classification	Supervised	Discrete class/label	Logistic regression, random forest, SVM	Flag an invoice as fraudulent / not fraudulent
Clustering	Unsupervised	Group assignment (no fixed labels)	K-Means, DBSCAN, hierarchical clustering	Segment customers by purchasing behavior

Supervised Classification with scikit-learn

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report
import pandas as pd

df = pd.read_csv("invoices.csv")
X = df[["amount", "days_to_due", "vendor_risk_score"]]
y = df["is_late_payment"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

model = RandomForestClassifier(n_estimators=200, max_depth=8)
model.fit(X_train, y_train)

preds = model.predict(X_test)
print(classification_report(y_test, preds))
```

EXPECTED OUTPUT

	precision	recall	f1-score
on_time	0.94	0.96	0.95
late	0.81	0.75	0.78

ENTERPRISE USE CASE

Flags high-risk invoices for early collections follow-up before they become overdue.

Feature Engineering

OBJECTIVE *Turn raw data into signals a model can learn from.*

- Feature engineering transforms raw fields into representations that expose predictive signal — often more impactful on final accuracy than the choice of algorithm.
- Common techniques: ratios and aggregates (amount-to-average-order ratio), time-based features (day-of-week, recency), categorical encoding (one-hot, target encoding), and text/embedding features.
- Domain knowledge matters most here: an Oracle DBA knows that 'AWR snapshot interval' is a meaningful engineered feature for a performance-anomaly model; a generic data scientist may not.
- Deep Learning (Module 3) automates much of this via representation learning — but structured enterprise data (ERP, financial) still benefits heavily from hand-engineered features.

Model Evaluation

OBJECTIVE *Choose the right metric for the task — accuracy alone is often misleading.*

- **Classification:**

Accuracy, Precision, Recall, F1-score, ROC-AUC. Precision/Recall matter more than accuracy on imbalanced data (e.g., 1% fraud rate).

- **Regression:**

Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), R^2 — RMSE penalizes large errors more heavily than MAE.

- **Validation methodology:**

K-fold cross-validation gives a more reliable performance estimate than a single train/test split, especially on smaller enterprise datasets.

GOVERNANCE NOTE

Enterprise model evaluation must also report performance by segment (region, customer tier) to catch fairness and reliability gaps a single aggregate score can hide.

Overfitting and Underfitting

OBJECTIVE *Diagnose the two failure modes of model fitting and know the fix for each.*

- **Overfitting:**

Model memorizes training data — including noise — and performs well in training but poorly on unseen data. Fix: more data, regularization, simpler model, early stopping, dropout (Module 3).

- **Underfitting:**

Model is too simple to capture the underlying pattern and performs poorly even on training data. Fix: more expressive model, better features, longer training.

- The goal is the sweet spot on the bias-variance tradeoff: low enough bias to fit real patterns, low enough variance to generalize.

ENTERPRISE EXAMPLE

A demand-forecasting model with 100% training accuracy but poor live performance is a textbook overfitting signal — validated via a held-out validation-vs-training accuracy gap.

Machine Learning — Enterprise Use Cases

OBJECTIVE *Anchor ML concepts in recurring Oracle-relevant business problems.*

- Financial forecasting and variance analysis in EPM/FP&A processes.
- Credit risk scoring and collections prioritization in Order-to-Cash.
- Predictive maintenance on manufacturing equipment using sensor telemetry (regression/classification).
- Customer churn prediction and next-best-action recommendation in CX platforms.
- Supply-chain demand forecasting and inventory optimization.
- Database workload classification for Autonomous Database's automated tuning decisions.

MODULE

03

Deep Learning

Learning Objectives

- Differentiate Machine Learning from Deep Learning
- Name every component of an artificial neuron and explain its role
- Trace forward propagation, loss computation, and backpropagation
- Explain epoch, batch, and learning rate as training controls

ML vs. DL

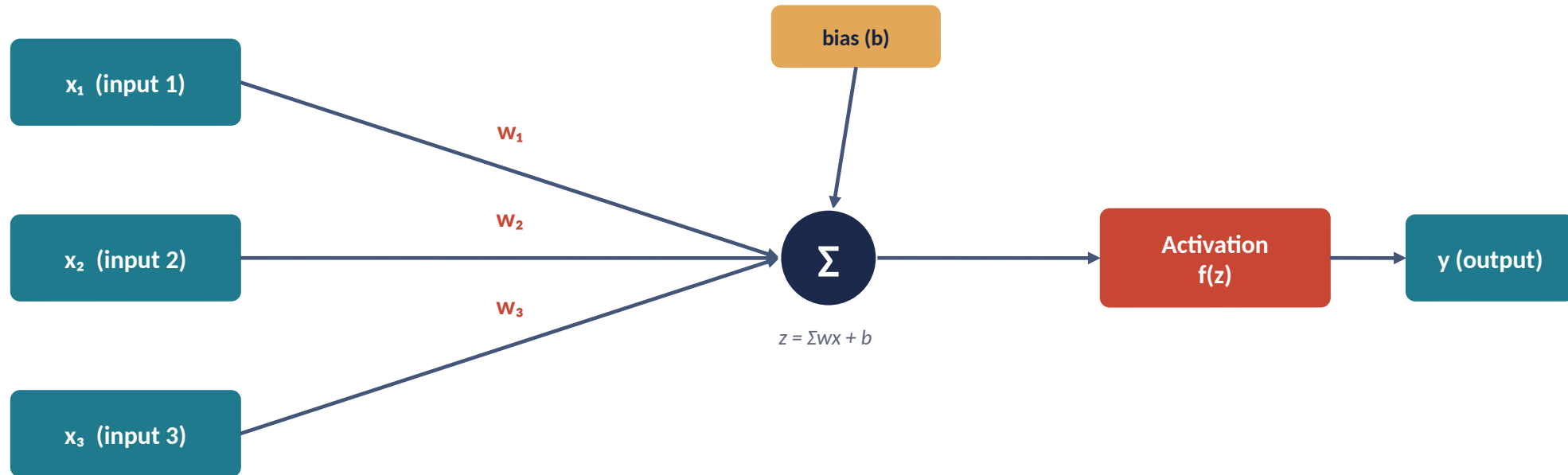
Dimension	Machine Learning	Deep Learning
Feature engineering	Manual — a human designs predictive features	Automatic — network learns hierarchical features from raw data
Data requirement	Effective on small-to-medium structured datasets	Needs large datasets to outperform classical ML
Compute requirement	Modest — CPU is often sufficient	High — typically requires GPU/TPU acceleration
Interpretability	Higher (trees, linear models are explainable)	Lower — deep nets are largely black-box
Typical data types	Structured/tabular data	Images, audio, text, video, and also large-scale tabular data
Representative models	Linear/logistic regression, random forest, XGBoost	CNN, RNN, LSTM, Transformer

Why Deep Learning: Neural Networks

OBJECTIVE *Understand what a neural network is and why depth matters.*

- A neural network is a computational model composed of layers of interconnected units (neurons) that jointly learn a function mapping inputs to outputs.
- "Deep" refers to having multiple hidden layers between input and output — depth lets the network build increasingly abstract representations (edges → shapes → objects, in an image model).
- Unlike classical ML, a deep network learns its own features directly from raw pixels, audio samples, or token sequences, removing the manual feature-engineering step from Module 2.
- The remainder of this module dissects a single neuron's math, then the training loop (forward pass, loss, backpropagation, gradient descent) that fits millions of parameters simultaneously.

Anatomy of an Artificial Neuron



Every neuron performs the same two steps: a weighted linear sum, then a non-linear activation.

How a Single Neuron Computes Its Output

- 1 **Inputs (x):** Numeric feature values feeding the neuron — e.g., normalized sensor readings or upstream layer activations.
- 2 **Weights (w):** One learnable parameter per input, controlling how strongly that input influences the neuron. Learned during training.
- 3 **Bias (b):** A learnable offset added to the weighted sum, letting the neuron shift its activation threshold independent of the inputs.
- 4 **Weighted sum (z):** $z = w_1x_1 + w_2x_2 + w_3x_3 + b$ — a linear combination of inputs, weights, and bias.
- 5 **Activation f(z):** A non-linear function applied to z. Without it, stacking layers would collapse into one linear function — non-linearity is what lets networks learn complex patterns.
- 6 **Output (y):** $y = f(z)$ is passed forward as input to the next layer, or is the network's final prediction.

Common Activation Functions

Function	Formula	Range	Typical Use
Sigmoid	$f(z) = 1 / (1 + e^{-z})$	(0, 1)	Binary classification output layer
Tanh	$f(z) = (e^z - e^{-z}) / (e^z + e^{-z})$	(-1, 1)	Hidden layers needing zero-centered output
ReLU	$f(z) = \max(0, z)$	$[0, \infty)$	Default for hidden layers — fast, avoids vanishing gradients
Leaky ReLU	$f(z) = z$ if $z > 0$ else αz	$(-\infty, \infty)$	ReLU variant fixing the “dying ReLU” problem
Softmax	$f(z)_i = e^{z_i} / \sum e^{z_j}$	(0, 1), sums to 1	Multi-class classification output layer

Forward Propagation and the Loss Function

OBJECTIVE *Understand how a network turns input into a prediction, and how that prediction is scored.*

- Forward propagation is the left-to-right pass of data through every layer: each layer computes $z = Wx + b$ then applies its activation, feeding the result to the next layer, until the output layer produces a prediction.
- A loss function quantifies how wrong that prediction is compared to the true label — training's entire objective is to minimize this value.

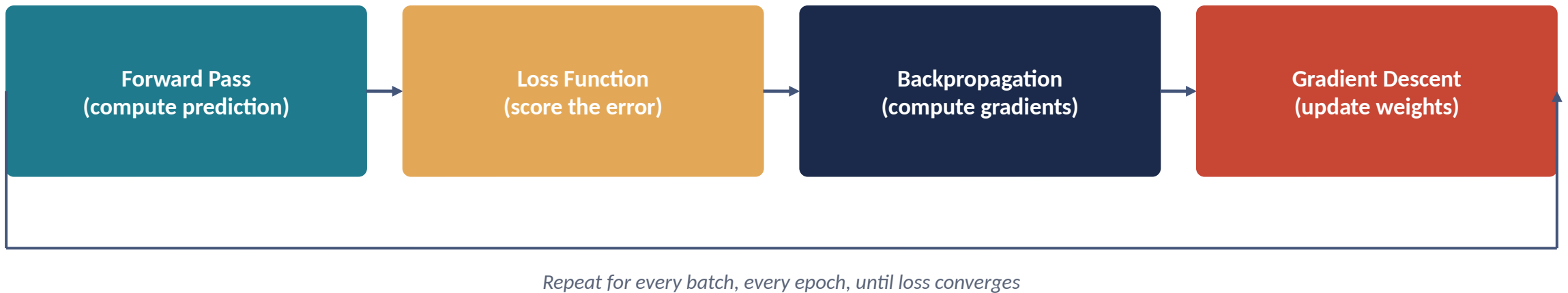
Mean Squared Error (MSE): used for regression — penalizes larger errors quadratically.

Cross-Entropy Loss: used for classification — penalizes confident wrong predictions heavily.

- The loss value computed here is exactly what backpropagation (next slide) works backward from to update every weight in the network.

The Neural Network Training Loop

Backpropagation applies the chain rule to compute how much each weight contributed to the loss; gradient descent then nudges each weight opposite its gradient.



One training step: a forward pass produces a prediction, the loss scores it, and backpropagation + gradient descent update every weight before repeating.

Epoch, Batch, and Learning Rate

OBJECTIVE Master the three hyperparameters that govern every training run.

- **Epoch:**

One complete pass through the entire training dataset. Training typically runs for many epochs (e.g., 20–100) until loss plateaus.

- **Batch:**

A subset of training examples processed together before one weight update. Batch size trades memory/speed against gradient stability — common sizes: 32, 64, 128.

- **Learning Rate:**

The step size used in gradient descent's weight update. Too high overshoots the minimum and diverges; too low converges very slowly. Typically $1e-2$ to $1e-5$, often scheduled to decay over training.

RULE OF THUMB

1 epoch = (dataset size / batch size) weight-update steps. A 10,000-row dataset with batch size 100 performs 100 updates per epoch.

A Minimal Neural Network with Keras

```
import tensorflow as tf
from tensorflow.keras import layers, models

model = models.Sequential([
    layers.Dense(16, activation="relu", input_shape=(6,)),
    layers.Dense(8, activation="relu"),
    layers.Dense(1, activation="sigmoid")
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss="binary_crossentropy", metrics=["accuracy"])

history = model.fit(
    X_train, y_train,
    epochs=30, batch_size=64,
    validation_data=(X_val, y_val))
```

EXPECTED OUTPUT

```
Epoch 28/30 - loss: 0.184 - accuracy: 0.932
Epoch 29/30 - loss: 0.181 - accuracy: 0.934
Epoch 30/30 - loss: 0.179 - accuracy: 0.935
```

ENTERPRISE USE CASE

A binary classifier (e.g., equipment-failure prediction) trained on engineered sensor features.

MODULE

04

Neural Networks and ANN

Learning Objectives

- Connect the biological neuron analogy to the artificial neuron
- Write the mathematical representation of a dense layer
- Identify input, hidden, output, and dense layers in an ANN
- Walk the complete ANN training workflow end to end

The Biological Neuron Analogy

OBJECTIVE *Ground the artificial neuron in the biological metaphor that inspired it.*

- A biological neuron receives electrochemical signals through dendrites, integrates them in the cell body (soma), and fires an output signal along its axon once a threshold is exceeded — synapses weight each incoming signal's influence.
- The artificial neuron is a deliberately simplified mathematical abstraction of this process — not a biologically accurate simulation.
- The analogy is pedagogically useful but should not be over-extended: real biological learning (synaptic plasticity) differs mechanically from gradient descent.

BIOLOGICAL → ARTIFICIAL

Dendrites → inputs (x). Synaptic strength → weights (w). Soma integration → weighted sum (z). Firing threshold → activation function $f(z)$. Axon output → y .

The Artificial Neuron — Mathematical Representation

OBJECTIVE Write the neuron's computation formally, in scalar and vector form.

- **Scalar form (n inputs):** $z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$, $y = f(z)$
- **Vector form:** $z = w^T x + b$, $y = f(z)$ — where w and x are n -dimensional vectors and b is a scalar.
- For an entire dense layer of m neurons processing n inputs, this generalizes to matrix form: $Z = XW + b$, where W is an $(n \times m)$ weight matrix and b is a length- m bias vector — every neuron's weighted sum is computed in a single matrix multiplication.
- This matrix formulation is exactly what frameworks like TensorFlow and PyTorch execute on GPU/TPU hardware — it is why deep learning scales with parallel compute.

ANN Architecture: Layers

OBJECTIVE *Name each layer type in a fully connected Artificial Neural Network.*

- **Input layer:**

Holds the raw feature vector — one node per input feature. Performs no computation of its own.

- **Hidden layer(s):**

Intermediate layers that learn increasingly abstract representations. “Deep” means more than one hidden layer.

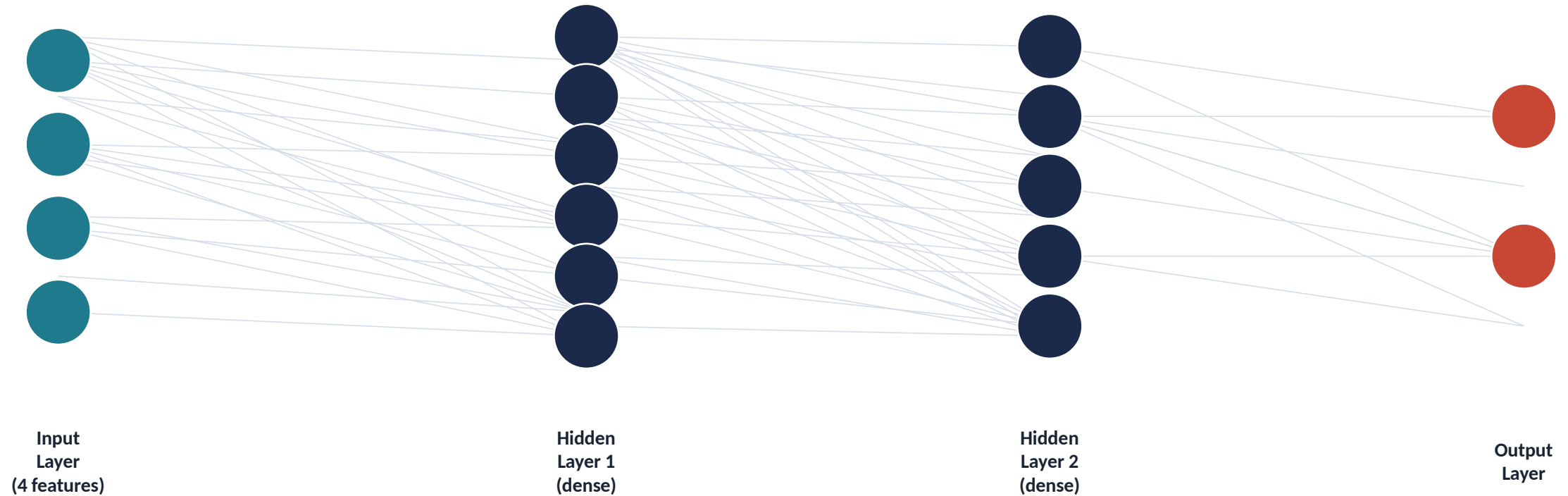
- **Output layer:**

Produces the final prediction — one neuron for regression/binary tasks, or one per class (with softmax) for multi-class classification.

- **Dense (fully connected) layer:**

A layer in which every neuron connects to every neuron in the previous layer — the default ANN layer type, in contrast to CNN's local connectivity (Module 5).

ANN Architecture — Fully Connected Network



A fully connected (dense) feed-forward ANN: every neuron connects to every neuron in the adjacent layer.

Reading the ANN Architecture

- 1 **Input layer (4 nodes):** Each node holds one numeric feature of a single record — no computation occurs here.
- 2 **Hidden Layer 1 (6 neurons, dense):** Every neuron receives all 4 inputs, computes its own weighted sum + bias, and applies ReLU.
- 3 **Hidden Layer 2 (5 neurons, dense):** Receives all 6 activations from Hidden Layer 1 — deeper layers combine lower-level features into higher-level ones.
- 4 **Output layer (2 neurons):** Produces class scores (e.g., via softmax) for a 2-class classification problem.
- 5 **Parameter count:** This network has $(4 \times 6 + 6) + (6 \times 5 + 5) + (5 \times 2 + 2) = 30 + 35 + 35 + 12 = 71$ learnable parameters — every line and every bias is one trainable weight.

The Complete ANN Training Workflow

OBJECTIVE *See ANN training as one disciplined end-to-end pipeline.*

- 1. Define architecture — choose number of hidden layers, neurons per layer, and activation functions based on problem complexity.
- 2. Initialize weights — small random values (e.g., He or Xavier initialization) to break symmetry between neurons.
- 3. Forward propagate a batch — compute predictions layer by layer (Module 3).
- 4. Compute loss — compare predictions to true labels.
- 5. Backpropagate — compute the gradient of the loss with respect to every weight via the chain rule.
- 6. Update weights — apply gradient descent (or Adam/RMSProp) scaled by the learning rate.
- 7. Repeat steps 3–6 for every batch, across every epoch, monitoring validation loss to catch overfitting.
- 8. Evaluate on the held-out test set, then deploy behind an inference endpoint.

Inspecting an ANN's Architecture and Parameter Count

```
from tensorflow.keras import layers, models

model = models.Sequential([
    layers.Input(shape=(4,)),
    layers.Dense(6, activation="relu", name="hidden_1"),
    layers.Dense(5, activation="relu", name="hidden_2"),
    layers.Dense(2, activation="softmax", name="output")
])
model.summary()
```

EXPECTED OUTPUT

Layer (type)	Output Shape	Param #
hidden_1 (Dense)	(None, 6)	30
hidden_2 (Dense)	(None, 5)	35
output (Dense)	(None, 2)	12
Total params: 77		Trainable params: 77

ENTERPRISE USE CASE

Verifying architecture and parameter budget before training on constrained OCI Data Science compute shapes.

MODULE

05

Convolutional Neural Networks (CNN)

Learning Objectives

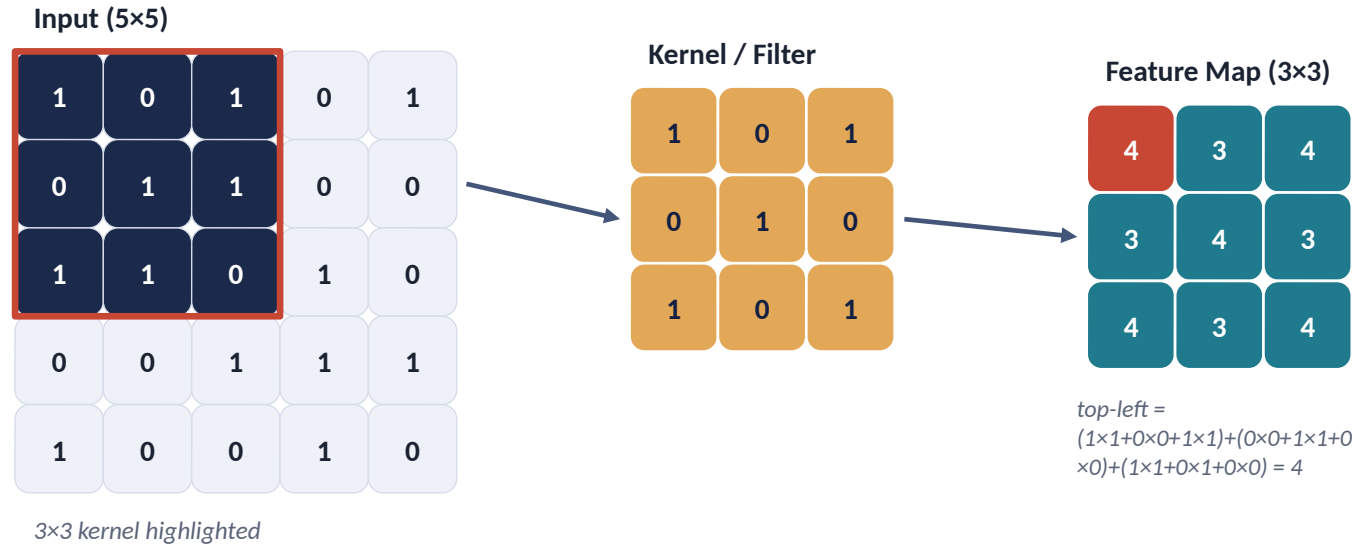
- Explain why CNNs outperform ANNs on image and spatial data
- Define convolution, kernel, feature map, stride, and padding
- Explain ReLU, pooling, flattening, and the fully connected classifier
- Map CNNs to enterprise OCR, inspection, and document-classification use cases

Why CNN?

OBJECTIVE *Motivate convolutional architecture as a fix for ANN's weaknesses on images.*

- A fully connected ANN applied to a 224×224 RGB image would need over 150,000 input neurons, and a single dense hidden layer would require tens of millions of weights — computationally infeasible and prone to severe overfitting.
- Dense layers also discard spatial structure: shuffling an image's pixels would not change an ANN's computation, even though it destroys the image.
- CNNs solve both problems with two ideas: local connectivity (each neuron looks at a small spatial region, not the whole image) and parameter sharing (the same small filter is reused across every position in the image).
- This makes CNNs translation-invariant — a learned filter for “edge” or “defect” detects that pattern anywhere in the image.

The Convolution Operation



A 3×3 kernel slides across the input, computing one dot product per position to produce the feature map.

Stride and Padding

OBJECTIVE *Control how the kernel moves and how borders are handled.*

- **Stride:**

The number of pixels the kernel moves between applications. Stride 1 moves one pixel at a time (dense overlap, larger feature map); stride 2 skips a pixel each step (smaller, coarser feature map, less compute).

- **Padding:**

“Valid” padding uses no border padding — the feature map shrinks each layer. “Same” padding adds zero-pixels around the border so the output stays the same spatial size as the input, preserving edge information across many layers.

- **Output size formula: $\text{out} = \lfloor (\text{in} - \text{kernel} + 2 \times \text{padding}) / \text{stride} \rfloor + 1$**

ENTERPRISE EXAMPLE

A defect-detection CNN on 4K manufacturing camera frames typically uses stride-2 early layers to reduce compute, then stride-1 deeper layers to preserve fine detail around a defect region.

ReLU and Pooling

OBJECTIVE *Add non-linearity and reduce spatial dimensionality between conv layers.*

- **ReLU activation:**

Applied element-wise to every feature map after convolution: $f(z) = \max(0, z)$. Introduces non-linearity and zeroes out negative activations, which also sparsifies the representation.

- **Pooling (down-sampling):**

Max pooling takes the maximum value in each small window (e.g., 2×2), reducing spatial size while keeping the strongest signal — makes the network more robust to small translations and reduces compute for deeper layers.

Average pooling (less common) takes the mean instead of the max — used when smoother down-sampling is preferred.

- A typical CNN block repeats: Convolution $\rightarrow$ ReLU $\rightarrow$ Pooling, several times, each block learning progressively higher-level visual features.

Flattening and the Fully Connected Layer

OBJECTIVE *Understand how spatial feature maps become a final classification.*

- After the final convolution/pooling block, the output is a stack of small feature maps (e.g., $7 \times 7 \times 128$) — still spatially structured.
- **Flattening:**

Reshapes this 3-D stack into a single 1-D vector (e.g., $7 \times 7 \times 128 = 6,272$ values), discarding spatial layout to hand off to dense layers.

- **Fully connected (dense) layer:**

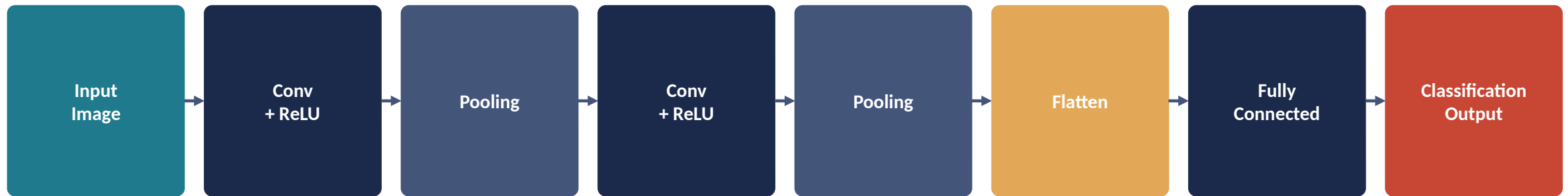
One or more standard ANN dense layers (Module 4) combine the flattened features into a final decision.

- **Classification:**

The final dense layer uses softmax to output a probability per class — the predicted class is the highest-probability output.

Complete CNN Architecture

Feature extraction (convolution + pooling blocks) → Classification (flatten + dense)



Convolution + ReLU + Pooling blocks extract features; flattening and dense layers classify them.

Data Flow Through the CNN

- 1 Input image:** A raw pixel grid (e.g., $224 \times 224 \times 3$ for RGB) enters the network.
- 2 Conv + ReLU (block 1):** Learned filters detect low-level features — edges, corners, color gradients.
- 3 Pooling (block 1):** Down-samples the feature maps, keeping the strongest activations and reducing compute.
- 4 Conv + ReLU / Pooling (block 2):** A second block combines low-level features into higher-level patterns — textures, shapes, parts.
- 5 Flatten:** The final feature-map stack is reshaped into a single feature vector.
- 6 Fully connected + Classification:** Dense layers combine the feature vector into class probabilities via softmax.

CNN — Enterprise Examples

OBJECTIVE *Connect CNN capability to concrete enterprise and Oracle-relevant workloads.*

- **OCR (Optical Character Recognition):**

CNN-based feature extraction underlies text detection/recognition — the vision layer behind OCI Document Understanding.

- **Document classification:**

Automatically route incoming scanned documents (invoice, PO, contract, ID) by visual layout, not just text content.

- **Image inspection:**

Visual quality checks on assembly-line photos — surface scratches, missing components, misalignment.

- **Defect detection:**

Manufacturing quality control — classify or localize defects in real time from line-camera feeds, reducing manual inspection cost and escape rate.

ENTERPRISE EXAMPLE

A PCB manufacturer runs a CNN defect classifier on line-camera images, flagging solder-joint defects before final assembly — directly analogous to ABC-style manufacturing QA workflows.

A Small CNN for Image Classification

```
from tensorflow.keras import layers, models

model = models.Sequential([
    layers.Input(shape=(128, 128, 3)),
    layers.Conv2D(32, (3,3), activation="relu", padding="same"),
    layers.MaxPooling2D((2,2)),
    layers.Conv2D(64, (3,3), activation="relu", padding="same"),
    layers.MaxPooling2D((2,2)),
    layers.Flatten(),
    layers.Dense(64, activation="relu"),
    layers.Dense(2, activation="softmax") # defect / no-defect
])
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
```

EXPECTED OUTPUT

```
Model: "sequential"
Total params: 1,655,682
Trainable params: 1,655,682
```

ENTERPRISE USE CASE

Binary defect classifier trained on line-camera images for automated visual QA.

MODULE

06

Recurrent Neural Networks (RNN)

Learning Objectives

- Explain why sequential data needs a different architecture than CNN/ANN
- Describe the RNN hidden state and its recurrent connection
- Read an unrolled RNN diagram and trace forward propagation through time
- Explain the vanishing-gradient problem and RNN's practical limitations

Sequential Data and Why RNN

OBJECTIVE *Motivate a memory-carrying architecture for ordered data.*

- Sequential data has order that carries meaning: text (word order), time series (timestamp order), audio, log streams, DNA sequences. Shuffling the order changes or destroys the meaning.
- CNNs and ANNs process each input independently with no notion of “what came before.” They cannot natively model dependencies across time steps.
- A Recurrent Neural Network (RNN) processes a sequence one element at a time while maintaining a hidden state — a running summary vector — that is updated at every step and carried forward to influence the next step.
- This gives RNNs memory: the output at time t can depend on everything the network has seen from time 0 through t .

ENTERPRISE EXAMPLE

Predicting the next likely error code in a stream of application log lines depends on the sequence of preceding log lines, not any single line in isolation.

RNN Architecture and the Hidden State

OBJECTIVE Define the recurrence relation at the heart of every RNN.

- At each time step t , the RNN cell takes two inputs: the current input x_t and the previous hidden state h_{t-1} .
- **Recurrence relation:** $h_t = f(W_h h_{t-1} + W_x x_t + b)$

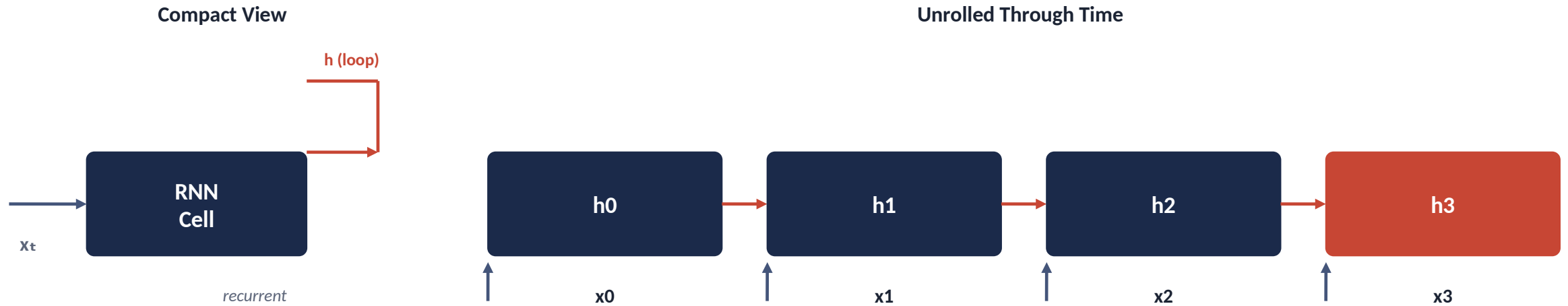
The same weight matrices (W_h , W_x) and bias are reused at every time step — this parameter sharing across time is what lets an RNN handle sequences of any length with a fixed parameter count.

- An output y_t can be produced at every step (sequence-to-sequence), or only at the final step (sequence-to-one) — e.g., a single sentiment label after reading a full review.

KEY INTUITION

The hidden state h_t is the network's compressed memory of everything seen so far — it is both an output of the current step and an input to the next.

RNN — Recurrent Cell and Unrolled View



The same cell (with shared weights) is applied at every time step; “unrolling” makes the time dimension explicit.

Forward Propagation Through Time

- 1 **t = 0:** $h_0 = f(W_x x_0 + b)$ — the initial hidden state, seeded from the first input (previous hidden state h_{-1} is typically zero-initialized).
- 2 **t = 1:** $h_1 = f(W_h h_0 + W_x x_1 + b)$ — now incorporates both the new input x_1 and the memory carried from h_0 .
- 3 **t = 2, 3, ...:** The same update repeats, so h_t always summarizes the entire prefix $x_0 \dots x_t$.
- 4 **Final step:** The last hidden state (or every h_t , depending on the task) feeds a final dense + softmax layer to produce a prediction.
- 5 **Weight sharing:** W_h , W_x , and b are identical copies at every unrolled step — only 3 parameter sets are trained, regardless of sequence length.

Vanishing Gradient and RNN Limitations

OBJECTIVE *Understand why plain RNNs struggle with long sequences.*

- **Vanishing gradient:**

During backpropagation through time, gradients are repeatedly multiplied by the same recurrent weight matrix at every step. If those values are < 1 , the gradient shrinks exponentially over many steps — early time steps receive almost no learning signal.

- **Practical consequence:**

A plain RNN effectively “forgets” information from more than ~10–20 steps in the past — it cannot learn long-range dependencies (e.g., connecting a pronoun to a noun 50 words earlier).

Exploding gradient (the opposite failure) can also occur when weights are > 1 , causing unstable, diverging training — mitigated with gradient clipping.

- These limitations directly motivate LSTM (Module 7), which adds a dedicated memory pathway designed specifically to preserve gradients over long sequences.

A Simple RNN for Sequence Classification

```
from tensorflow.keras import layers, models

model = models.Sequential([
    layers.Input(shape=(50, 1)),      # 50 timesteps, 1 feature
    layers.SimpleRNN(32, activation="tanh"),
    layers.Dense(16, activation="relu"),
    layers.Dense(1, activation="sigmoid")
])
model.compile(optimizer="adam",
              loss="binary_crossentropy",
              metrics=["accuracy"])
model.fit(X_train, y_train, epochs=15, batch_size=32)
```

EXPECTED OUTPUT

Epoch 15/15 - loss: 0.241 - accuracy: 0.901

ENTERPRISE USE CASE

Classifying a rolling 50-reading sensor window as normal vs. pre-failure on a production line.

MODULE

07

Long Short-Term Memory (LSTM)

Learning Objectives

- Explain why LSTM was introduced to fix RNN's vanishing-gradient problem
- Name the cell state and all three gates: forget, input, output
- Trace LSTM's data flow step by step through one time step
- Apply LSTM to forecasting, log analysis, and predictive maintenance

Why LSTM Was Introduced

OBJECTIVE *Connect LSTM's design directly to the RNN limitation from Module 6.*

- Hochreiter & Schmidhuber (1997) introduced LSTM specifically to solve the vanishing-gradient problem that prevents plain RNNs from learning long-range dependencies.
- The key idea: add a separate cell state that runs through the sequence with only minor, mostly-linear modifications at each step — instead of being repeatedly squashed through a non-linearity like the RNN hidden state.
- Three learned gates (forget, input, output) control what information enters, leaves, and is exposed from that cell state at every time step — the network learns what to remember and what to discard.
- This lets an LSTM preserve gradient flow — and therefore memory — across hundreds of time steps, versus roughly 10–20 for a plain RNN.

Cell State and the Three Gates

OBJECTIVE *Preview each component before seeing the full data-flow diagram.*

- **Cell state (C_t):**

The LSTM's long-term memory — a vector that flows across time steps with only gated, mostly-additive updates.

- **Forget gate (f_t):**

A sigmoid layer deciding what fraction of the previous cell state C_{t-1} to keep vs. discard.

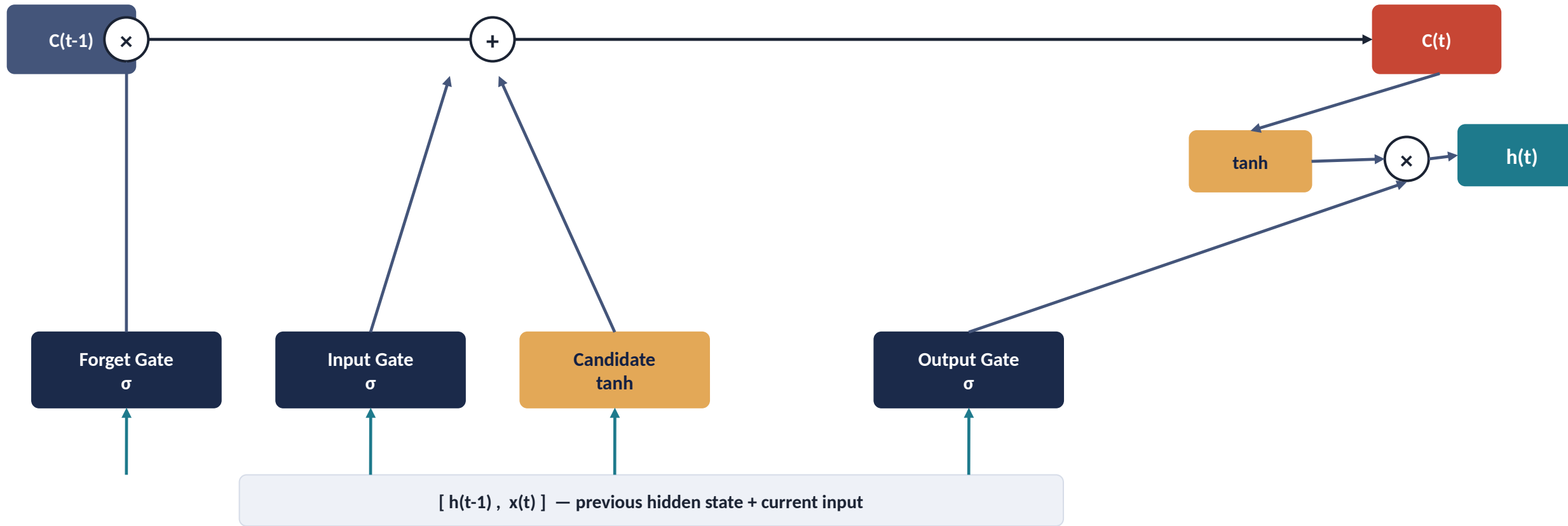
- **Input gate (i_t) + candidate state ($\tilde{C}_t$):**

The input gate decides how much new information to add; the candidate (a tanh layer) proposes what that new information is.

- **Output gate (o_t):**

Decides what part of the (updated) cell state is exposed as the hidden state h_t , which is also the step's output.

LSTM Cell — Gates and Cell-State Flow



The cell state runs along the top with light, gated modifications; three sigmoid gates and one tanh candidate control the flow.

LSTM Data Flow — Step by Step

1. **Shared input:** The previous hidden state $h(t-1)$ and current input $x(t)$ are concatenated and fed to all four gates/candidate.
2. **Forget gate:** $f(t) = \sigma(W_r[h(t-1), x(t)] + b_r)$ outputs values in $(0,1)$ per cell-state dimension — near 0 erases, near 1 keeps.
3. **Input gate + candidate:** $i(t) = \sigma(\dots)$ decides how much to write; $\tilde{C}(t) = \tanh(\dots)$ proposes new candidate values to write.
4. **Cell state update:** $C(t) = f(t) \times C(t-1) + i(t) \times \tilde{C}(t)$ — forget part of the old memory, add part of the new candidate.
5. **Output gate:** $o(t) = \sigma(\dots)$ decides what portion of the updated cell state to expose.
6. **Hidden state:** $h(t) = o(t) \times \tanh(C(t))$ — this is both the step's output and the h passed to the next time step.

LSTM — Enterprise Examples

OBJECTIVE *Map LSTM's long-range memory to concrete sequence-modeling workloads.*

- **Time-series forecasting:**

Revenue, demand, or capacity forecasting where recent and seasonal history both matter.

- **Log analysis:**

Modeling sequences of system/application log events to flag anomalous event orderings before an outage.

- **Text processing:**

Sequence labeling and classification on text prior to the Transformer era — still used in lightweight, low-latency NLP components.

- **Predictive maintenance:**

Learning degradation patterns across long sensor-reading histories to forecast equipment failure ahead of time.

ENTERPRISE EXAMPLE

An LSTM trained on 30-day rolling vibration-sensor sequences from CNC machines predicts bearing failure 5–7 days in advance, enabling scheduled rather than emergency maintenance.

LSTM for Time-Series Forecasting

```
from tensorflow.keras import layers, models

model = models.Sequential([
    layers.Input(shape=(30, 4)),      # 30-day window, 4 sensor features
    layers.LSTM(64, return_sequences=True),
    layers.LSTM(32),
    layers.Dense(16, activation="relu"),
    layers.Dense(1)                  # predicted failure risk score
])
model.compile(optimizer="adam", loss="mse", metrics=["mae"])
model.fit(X_train, y_train, epochs=25,
          batch_size=64, validation_split=0.2)
```

EXPECTED OUTPUT

```
Epoch 25/25 - loss: 0.0132 - mae: 0.081
val_loss: 0.0169 - val_mae: 0.093
```

ENTERPRISE USE CASE

Forecasting next-week bearing-failure risk from a rolling multivariate sensor window.

ANN vs. CNN vs. RNN vs. LSTM

Architecture	Best For	Key Structural Idea	Weakness
ANN	Structured/tabular data	Fully connected dense layers	No spatial or sequential awareness
CNN	Images, spatial grids	Local receptive fields + shared filters	Not suited to variable-length sequences
RNN	Short sequences	Recurrent hidden state carried across steps	Vanishing gradient limits long-range memory
LSTM	Long sequences	Gated cell state (forget/input/output gates)	Sequential computation — slow to train/parallelize

The Transformer Architecture

Learning Objectives

- Explain why Transformers replaced RNN/LSTM for sequence modeling
- Name every Transformer sublayer: attention, FFN, residuals, layer norm
- Compute self-attention conceptually from Query, Key, and Value
- Explain multi-head attention and positional encoding

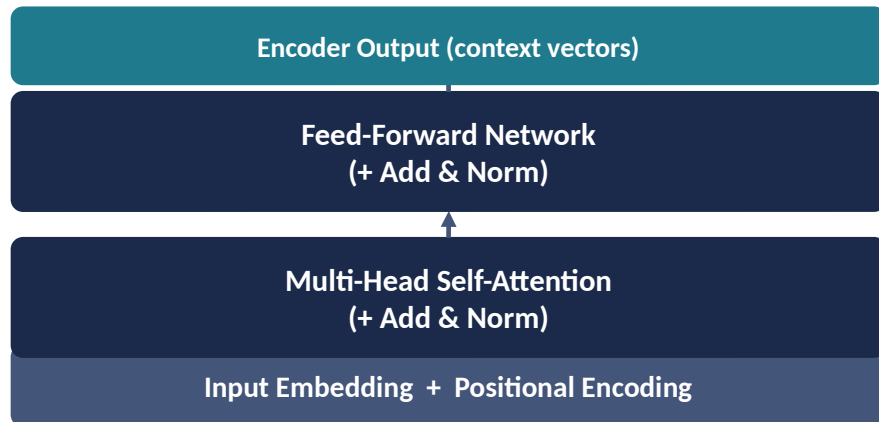
Why Transformers?

OBJECTIVE *Motivate the architecture that replaced RNN/LSTM as the default for sequence modeling.*

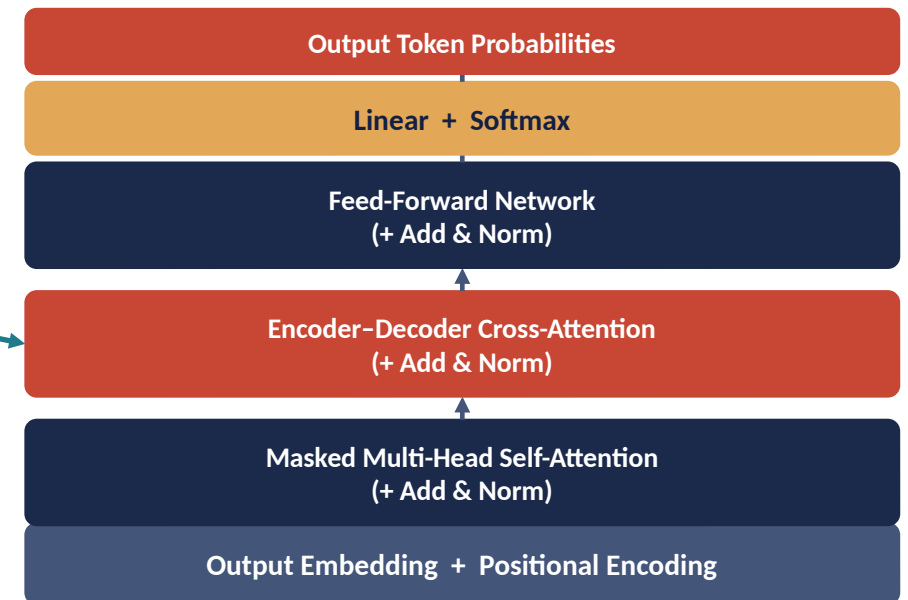
- RNNs and LSTMs process a sequence strictly step by step — step t must finish before step $t+1$ can start. This sequential dependency prevents parallelization across time and makes training on long sequences slow.
- Even LSTM's gated memory degrades over very long sequences (hundreds to thousands of tokens) — it approximates long-range dependency, it does not solve it.
- The 2017 paper “Attention Is All You Need” introduced the Transformer: it discards recurrence entirely and instead lets every position attend directly to every other position in the sequence, in parallel, via a mechanism called self-attention.
- Result: full GPU parallelization during training, and direct, undiluted access from any token to any other token regardless of distance — the architectural foundation of every modern LLM.

Transformer Architecture — Encoder & Decoder

ENCODER



DECODER



The encoder builds contextual representations of the input; the decoder generates output tokens, attending to both its own prior output and the encoder's output.

Reading the Encoder–Decoder Stack

- 1 Embeddings + positional encoding:** Input and output tokens are embedded into vectors, then positional information is added since Transformers have no inherent sense of order.
- 2 Encoder self-attention:** Every input token attends to every other input token, building a context-aware representation of the full input sequence.
- 3 Decoder masked self-attention:** Each output position attends only to earlier output positions (masking prevents “seeing the future” during generation).
- 4 Cross-attention:** The decoder's Query attends to the encoder's Key/Value output — this is how the decoder grounds its generation in the input sequence.
- 5 Feed-forward + Add & Norm:** Each sublayer output is added back to its input (residual connection) and normalized, repeated N times per stack.
- 6 Linear + Softmax:** The final decoder representation is projected to vocabulary size and turned into a probability distribution over the next token.

Tokenization

OBJECTIVE *Understand how raw text becomes model-ready integers.*

- Tokenization splits raw text into subword units (tokens) using a fixed vocabulary learned from a large corpus — typically via Byte-Pair Encoding (BPE) or SentencePiece.
- Subword tokenization balances vocabulary size against sequence length: common words stay whole (“the”), rare words split into pieces (“tokeniz” + “ation”), and any string — including unseen words and typos — can still be represented.
- Each token maps to an integer ID; the model only ever sees integers, never raw characters.
- Tokenization directly drives cost and context-window usage in production LLM APIs, including OCI Generative AI — billing and context limits are measured in tokens, not words or characters.

ENTERPRISE EXAMPLE

“invoice reconciliation” might tokenize as ["invoice", " recon", "cil", "iation"] — 4 tokens for 2 words.

Embeddings

OBJECTIVE *Understand the dense vector representation every token is converted into.*

- An embedding layer maps each token ID to a dense, learned vector (e.g., 768 or 4,096 dimensions) that captures semantic meaning — tokens used in similar contexts end up with similar vectors.
- Embeddings are learned jointly with the rest of the model during pretraining; they are not fixed or hand-designed.
- The same underlying concept — mapping meaning to a dense vector — reappears in Module 9 as the basis for semantic search and Retrieval-Augmented Generation, though document/sentence embeddings there are typically produced by a dedicated embedding model rather than a generative model's internal layer.
- Vector arithmetic on embeddings often captures relationships: the classic example is $\text{vector}(\text{"king"}) - \text{vector}(\text{"man"}) + \text{vector}(\text{"woman"}) \approx \text{vector}(\text{"queen"})$.

Positional Encoding

OBJECTIVE *Understand how Transformers reintroduce order without recurrence.*

- Self-attention treats a sequence as an unordered set — computed relationships between token pairs don't depend on position. Without extra information, “the dog bit the man” and “the man bit the dog” would look identical to attention alone.
- Positional encoding injects position information by adding a position-dependent vector to each token embedding before the first layer.
- The original Transformer used fixed sinusoidal functions: $PE(pos, 2i) = \sin(pos / 10000^{(2i/d)})$, $PE(pos, 2i+1) = \cos(pos / 10000^{(2i/d)})$ — chosen so relative positions are linearly related, and the encoding generalizes to sequence lengths unseen during training.
- Many modern LLMs instead use learned or relative positional encodings (e.g., RoPE) — the sinusoidal scheme remains the clearest one to teach the underlying concept.

Self-Attention: Query, Key, and Value

OBJECTIVE *Introduce the three learned projections behind every attention computation.*

- For every token, three vectors are computed by multiplying its embedding by three learned weight matrices: Query (Q), Key (K), and Value (V).

- **Query:**

represents “what this token is looking for” in other tokens.

- **Key:**

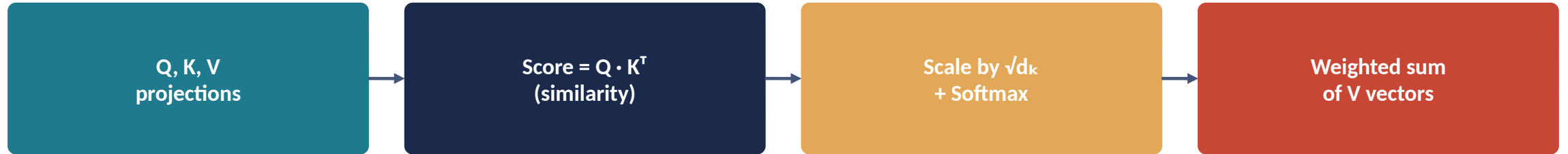
represents “what this token offers” to be matched against queries from other tokens.

- **Value:**

represents “the actual content” to pass along once a match is found.

- Attention score between two tokens = similarity between one token's Query and the other's Key (a dot product) — high similarity means “pay more attention to this token.”

Self-Attention Computation



$$\text{Attention}(Q, K, V) = \text{softmax}(Q K^T / \sqrt{d_k}) V$$

d_k = dimension of the Key vectors (the $\sqrt{d_k}$ scale keeps dot-product magnitudes stable before softmax).

Each token's Query is compared against every token's Key to produce attention weights, which blend the Value vectors into a context-aware output.

A Small Worked Self-Attention Example

Sentence: "The bank raised rates." Compute the new representation for the token "bank":

- 1 **1. Project:** "bank"'s embedding is multiplied by W^q to get its Query vector; every token's embedding (including "bank" itself) is multiplied by W_k and W_v to get Key and Value vectors.
- 2 **2. Score:** Dot-product "bank"'s Query against every token's Key: The(0.1), bank(0.9), raised(0.7), rates(0.8) — "raised" and "rates" score high because they disambiguate "bank" as financial, not a riverbank.
- 3 **3. Scale & softmax:** Scores are divided by $\sqrt{d_k}$ and passed through softmax, turning them into weights that sum to 1: e.g., 0.05, 0.40, 0.25, 0.30.
- 4 **4. Weighted sum:** "bank"'s new representation = $0.05 \times V(\text{The}) + 0.40 \times V(\text{bank}) + 0.25 \times V(\text{raised}) + 0.30 \times V(\text{rates})$ — a context-blended vector.
- 5 **Result:** "bank"'s output vector now encodes "financial institution," disambiguated purely from context, with no recurrence and no fixed window.

Multi-Head Attention

OBJECTIVE *Understand why Transformers run attention multiple times in parallel.*

- A single attention computation captures one type of relationship. Multi-head attention runs several attention operations (“heads”) in parallel, each with its own learned Q/K/V projection matrices.
- Different heads specialize: empirically, some heads learn syntactic relationships (subject-verb agreement), others learn coreference (pronoun-to-noun), others positional patterns.
- Head outputs are concatenated and passed through one more learned linear projection to produce the sublayer's final output — same dimensionality as a single-head equivalent, but richer.
- A typical LLM uses 8 to 96+ heads per layer, depending on model scale.

Feed-Forward, Residual Connections, and Layer Normalization

OBJECTIVE *Complete the Transformer block with its remaining structural components.*

- **Feed-forward network (FFN):**

A two-layer dense network (expand → activate → project back) applied identically to every position — it lets the model transform each token's representation independently after attention has mixed information across tokens.

- **Residual connection:**

Each sublayer's output is added back to its input ($\text{output} = \text{Sublayer}(x) + x$) — this preserves gradient flow through very deep stacks (12 to 100+ layers) and is what makes such depth trainable at all.

- **Layer normalization:**

Normalizes activations across the feature dimension after each residual add, stabilizing training and letting deep stacks converge reliably.

Scaled Dot-Product Attention in NumPy

```
import numpy as np

def softmax(x):
    e = np.exp(x - np.max(x, axis=-1, keepdims=True))
    return e / e.sum(axis=-1, keepdims=True)

def self_attention(Q, K, V):
    d_k = Q.shape[-1]
    scores = Q @ K.T / np.sqrt(d_k)      # similarity scores
    weights = softmax(scores)             # attention weights
    return weights @ V                    # context-blended output

# 4 tokens, embedding dim 8
Q = np.random.randn(4, 8); K = np.random.randn(4, 8); V = np.random.randn(4, 8)
context = self_attention(Q, K, V)
print(context.shape)
```

EXPECTED OUTPUT

(4, 8) # one context-aware vector per input token

ENTERPRISE USE CASE

Illustrates the exact computation performed millions of times per second inside OCI Generative AI's hosted foundation models.

RNN vs. LSTM vs. Transformer

Dimension	RNN	LSTM	Transformer
Sequence handling	Step-by-step, recurrent	Step-by-step, gated memory	Fully parallel via self-attention
Long-range dependency	Poor (vanishing gradient)	Good, up to ~hundreds of steps	Excellent — direct token-to-token access
Training speed	Slow (sequential)	Slow (sequential)	Fast — parallelizable on GPU/TPU
Positional awareness	Implicit (via order of processing)	Implicit (via order of processing)	Explicit — requires positional encoding
Dominant use today	Legacy / lightweight sequence tasks	Time-series, moderate-length sequences	LLMs, GenAI, virtually all state-of-the-art NLP

MODULE

09

Generative AI & RAG

Learning Objectives

- Define Generative AI, LLMs, and foundation models precisely
- Explain tokens, context window, prompts, and temperature
- Explain hallucination and how Retrieval-Augmented Generation mitigates it
- Trace the complete RAG workflow using Oracle Database Vector Search

What Is Generative AI?

OBJECTIVE *Define GenAI, LLMs, and foundation models with precision.*

- **Generative AI:**

Deep Learning models trained to generate new, original content — text, code, images, audio — rather than only classify or predict a fixed label.

- **Large Language Model (LLM):**

A Transformer-based (Module 8) neural network trained on massive text corpora to predict the next token in a sequence — this single objective, at scale, produces models capable of reasoning, summarization, translation, and code generation.

- **Foundation model:**

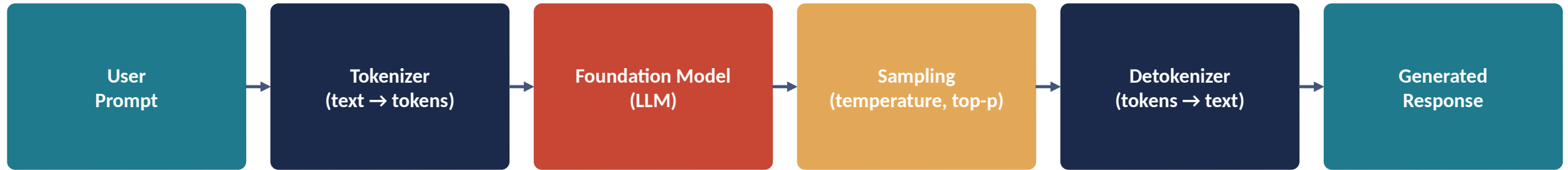
A large model pretrained on broad data, general-purpose enough to be adapted (via prompting or fine-tuning) to many downstream tasks without training from scratch — Cohere Command, Meta Llama, and GPT-family models are all foundation models.

ENTERPRISE EXAMPLE

OCI Generative AI provides managed, pay-per-use access to foundation models (Cohere, Meta Llama family) without customers hosting or training their own.

Generative AI System Architecture

Every generation call is autoregressive: token N depends on tokens 1..N-1.



The model repeats one step at a time: predict the next token's probability distribution, sample one token, append it, repeat — until an end-of-sequence token or the context window (Module 9) fills.

A prompt is tokenized, processed by the foundation model, and sampled token-by-token back into text.

Reading the GenAI System Architecture

1. **User prompt:** the raw text instruction, question, or conversation submitted to the system.
2. **Tokenizer:** converts the prompt's text into a sequence of integer token IDs the model can process (Module 8).
3. **Foundation model (LLM):** the Transformer-based network processes the token sequence and produces a probability distribution over the next possible token, repeated one token at a time.
4. **Sampling:** temperature and top-p settings determine how that probability distribution is turned into one chosen token — deterministic and safe at low temperature, more varied at high temperature.
5. **Detokenizer:** converts the generated token sequence back into human-readable text.
6. **Generated response:** returned to the caller once an end-of-sequence token is produced or the context window limit is reached.

Context Window and Prompt

OBJECTIVE *Understand the two hard limits and levers of every LLM interaction.*

- **Prompt:**

The text input provided to the model — instructions, context, and/or examples that condition its generated output. Module 10 covers prompt design in depth.

- **Context window:**

The maximum number of tokens (input + output combined) the model can process in a single call — e.g., 4K, 32K, or 128K tokens depending on the model. Content beyond the window is simply not visible to the model.

- Context window size directly bounds RAG design: retrieved passages, conversation history, and system instructions all compete for the same token budget.
- Longer context windows enable larger retrieved-document sets or longer conversation history, but increase latency and cost roughly with token count.

Temperature and Hallucination

OBJECTIVE *Control output randomness and understand the model's core reliability risk.*

- **Temperature:**

A sampling parameter (typically 0 to 2) controlling randomness. Near 0 = deterministic, most-likely-token output (best for SQL generation, factual Q&A). Higher values (0.7–1.0+) increase diversity and creativity, at the cost of consistency.

- **Hallucination:**

When a model generates fluent, confident, but factually incorrect or fabricated content — a fundamental consequence of the model predicting plausible tokens, not verified facts.

- **Mitigations:**

Lower temperature for factual tasks; grounding the model in retrieved, verifiable source documents (RAG, covered next); explicit “cite your source” prompting; and human review of high-stakes outputs.

ENTERPRISE EXAMPLE

An ungrounded LLM asked for an Oracle init parameter's default value may confidently state an incorrect number — a RAG system grounded in actual Oracle documentation prevents this.

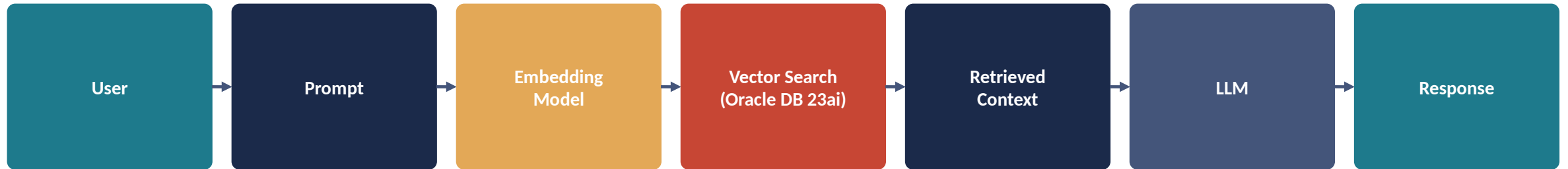
Retrieval-Augmented Generation (RAG)

OBJECTIVE *Understand why and how RAG grounds an LLM in real, current, enterprise data.*

- An LLM's knowledge is frozen at training time and limited to its pretraining corpus — it has no built-in awareness of your private enterprise data, and no way to cite sources.
- RAG solves both problems: before generating an answer, the system retrieves relevant passages from an external knowledge base and inserts them into the prompt as grounding context.
- This turns the LLM from a closed-book exam-taker into an open-book one — it answers from supplied evidence rather than parametric memory alone, sharply reducing hallucination and enabling citation.
- RAG requires no model retraining, making it the fastest and lowest-cost way to make an LLM enterprise-aware — contrast with fine-tuning, compared later in this module.

RAG Architecture — End-to-End Workflow

User → Prompt → Embedding → Vector Search → Retrieved Context → LLM → Response



Oracle Database 23ai's AI Vector Search stores embeddings alongside relational data, so retrieval and enterprise data live in one system.

The RAG Workflow, Step by Step

- 1. User submits a question:** e.g., “What is our warranty policy for defective CNC parts?”
- 2. Embed the query:** An embedding model converts the question into a dense vector capturing its semantic meaning.
- 3. Vector search:** Oracle Database 23ai's AI Vector Search finds the document chunks whose embeddings are nearest (by cosine similarity) to the query vector — semantic match, not keyword match.
- 4. Assemble context:** The top-k retrieved passages are inserted into the prompt template alongside the original question.
- 5. Generate:** The LLM (via OCI Generative AI) produces an answer grounded in the retrieved passages, ideally with citations.
- 6. Return response:** The grounded, source-traceable answer is returned to the user — dramatically reducing hallucination versus an ungrounded prompt.

Vector Databases and Embeddings for Retrieval

OBJECTIVE *Connect Module 2's embeddings to production retrieval infrastructure.*

- A vector database stores high-dimensional embedding vectors and supports fast approximate-nearest-neighbor (ANN) search — finding the k most semantically similar vectors to a query vector among millions or billions of records.
- Oracle Database 23ai's AI Vector Search adds a native VECTOR data type and vector-indexing/search operators directly inside Oracle Database — enterprises get semantic search over their existing relational data without standing up a separate specialized vector database.
- Document ingestion pipeline: source documents → chunking (split into passages) → embedding model → store vector + source text + metadata → index for search.
- Chunk size and overlap materially affect RAG quality: chunks too large dilute relevance; chunks too small lose context — this is a key tuning parameter in production RAG systems.

Fine-Tuning

OBJECTIVE *Understand the alternative (and complement) to RAG for customizing model behavior.*

- Fine-tuning continues training a pretrained foundation model on a smaller, task-specific labeled dataset, updating some or all of its weights to shift its behavior, tone, or task performance.
- Fine-tuning changes what the model knows how to do (style, format, domain jargon fluency); it is not an efficient way to inject large volumes of frequently changing factual knowledge — that is RAG's role.
- OCI Generative AI supports importing and fine-tuning custom models on dedicated AI clusters, then hosting them for inference alongside pretrained models.
- Production systems frequently combine both: a fine-tuned model for domain tone/format, with RAG supplying current factual grounding — the two techniques are complementary, not competing.

Embedding Text and Running a Vector Search Query

```
import oracledb, array
from oci.generative_ai_inference import GenerativeAiInferenceClient
from oci.generative_ai_inference.models import EmbedTextDetails, OnDemandServingMode

client = GenerativeAiInferenceClient(config=oci.config.from_file())
details = EmbedTextDetails(
    inputs=["What is our warranty policy for defective CNC parts?"],
    serving_mode=OnDemandServingMode(model_id="cohere.embed-english-v3.0"),
    compartment_id=compartment_id)
vec = client.embed_text(details).data.embeddings[0]

# Oracle Database 23ai AI Vector Search
cursor.execute("""
    SELECT chunk_text FROM kb_chunks
    ORDER BY VECTOR_DISTANCE(embedding, :qv, COSINE)
    FETCH FIRST 5 ROWS ONLY""",
    qv=array.array("f", vec))
top_chunks = [row[0] for row in cursor]
```

EXPECTED OUTPUT

```
['Section 4.2: Parts replaced under warranty must\nbe reported within
30 days of installation...', ...]
```

ENTERPRISE USE CASE

Retrieval step of an internal Oracle knowledge-base assistant for manufacturing operations staff.

RAG vs. Fine-Tuning

Dimension	RAG	Fine-Tuning
What it changes	What the model can see (retrieved context)	How the model behaves (its weights)
Best for	Fast-changing or large factual knowledge	Style, tone, task format, domain jargon
Freshness	As current as the knowledge base — near real-time	Frozen at fine-tuning time; needs retraining to update
Cost & effort	Lower — no training required, add documents to the index	Higher — requires labeled data and a training run
Explainability	High — retrieved passages can be cited	Lower — behavior change is embedded in weights
OCI mechanism	Oracle DB 23ai Vector Search + OCI Generative AI	OCI Generative AI custom model fine-tuning on dedicated AI clusters

MODULE

10

Prompt Engineering

Learning Objectives

- Structure prompts using Role, Context, Task, Constraints, Examples, and Format
- Apply zero-shot, few-shot, and chain-of-thought prompting appropriately
- Request structured output and ground responses in supplied evidence
- Rewrite 10 realistic Oracle engineering prompts from poor to production-grade

Anatomy of an Effective Prompt

OBJECTIVE *Learn the six components that turn a vague request into a reliable one.*

- **Role:**

Tell the model what expertise to adopt — “you are a senior Oracle DBA.” Sets vocabulary and depth.

- **Context:**

Supply the relevant background — schema, environment, prior conversation, constraints of the system.

- **Task:**

State precisely what output is needed — one unambiguous instruction, not a vague request.

- **Constraints:**

Bound the answer — dialect, length, performance requirements, what to avoid.

- **Examples:**

Show the desired input/output pattern (few-shot) when format matters.

- **Output format:**

Specify exactly how the response should be structured — JSON, a table, a code block only.

RULE OF THUMB

Every prompt example in this module is rewritten using this exact six-part structure — treat it as a checklist for every production prompt you write.

Zero-Shot vs. Few-Shot Prompting

OBJECTIVE *Choose the right amount of demonstration for the task.*

- **Zero-shot prompting:**

Ask the model to perform a task with instructions alone, no examples. Works well for tasks the model has broad pretraining exposure to (general Q&A, common code patterns).

- **Few-shot prompting:**

Provide 2–5 input/output example pairs before the real request. Dramatically improves consistency for tasks with a specific desired format, tone, or edge-case handling the model wouldn't infer on its own.

- **Trade-off:**

Few-shot examples consume context-window tokens (Module 9) and increase cost/latency — use them when format consistency materially matters, not by default.

ENTERPRISE EXAMPLE

A log-classification prompt with 3 example log-line → severity-label pairs produces far more consistent labels than instructions alone.

Chain-of-Thought, Structured Output, and Grounding

OBJECTIVE *Three techniques for more reliable, more usable model output.*

- **Chain-of-thought (high-level):**

Prompting the model to reason through a problem in steps before giving a final answer generally improves accuracy on multi-step tasks (math, multi-step troubleshooting). In production, the step-by-step reasoning is typically requested internally but not shown to end users — only the final, validated answer is surfaced.

- **Structured output:**

Explicitly request JSON, XML, or a defined schema (“respond only with valid JSON matching this schema: {...}”) — essential when the response feeds another system rather than a human reader.

- **Grounding:**

Instructing the model to answer only from supplied context (“using only the documentation excerpt below”) and to say “not found in the provided context” rather than guess — the prompt-level discipline that pairs with RAG (Module 9) to suppress hallucination.

SQL Generation

POOR PROMPT

Write SQL for orders.

IMPROVED PROMPT

Role: senior Oracle SQL developer. Context: table ORDERS(order_id, customer_id, order_date, status, total_amount). Task: return total revenue per month for 2025, orders with status='COMPLETED' only. Constraints: Oracle SQL dialect, use TO_CHAR for month formatting, order chronologically.

EXPECTED OUTPUT

```
SELECT TO_CHAR(order_date, 'YYYY-MM') AS order_month,
       SUM(total_amount) AS revenue
FROM   orders
WHERE  status='COMPLETED'
       AND order_date >= DATE '2025-01-01'
GROUP BY TO_CHAR(order_date, 'YYYY-MM')
ORDER BY 1;
```

WHY IT'S BETTER

Specifying schema, dialect, filters, and formatting removes ambiguity that would otherwise force a generic, possibly non-Oracle-compatible query.

PL/SQL Explanation

POOR PROMPT

Explain this PL/SQL.

IMPROVED PROMPT

Role: PL/SQL trainer explaining to a junior developer. Task: explain, line by line, what the following procedure does and why a BULK COLLECT is used instead of a row-by-row cursor loop. Output format: numbered explanation followed by a one-paragraph summary of the performance implication. [procedure pasted below]

EXPECTED OUTPUT

```
1. Declares a BULK COLLECT cursor... 2. Fetches in batches of
500 to limit PGA memory... Summary: BULK COLLECT reduces
context switches between SQL and PL/SQL engines, cutting
elapsed time significantly versus row-by-row processing.
```

WHY IT'S BETTER

Naming the audience and requesting a specific structure (line-by-line + summary) produces a teachable explanation instead of a generic paraphrase.

Java Code Review

POOR PROMPT

Review this Java code.

IMPROVED PROMPT

Role: senior Java reviewer enforcing our team's standards (null-safety, try-with-resources for JDBC, no raw SQL string concatenation). Task: review the class below for correctness, thread-safety, and SQL-injection risk. Output format: a table with columns Issue | Severity | Line | Suggested Fix.

EXPECTED OUTPUT

```
| Issue | Severity | Line | Suggested Fix |  
|SQL built via string concat | High | 42 | Use  
PreparedStatement |  
|Connection not closed on exception | Medium | 51 | try-with-  
resources |
```

WHY IT'S BETTER

Defining the standards to check against and the exact output table produces an actionable, consistently formatted review instead of free-form prose.

Python Code Generation

POOR PROMPT

Write Python to read a CSV.

IMPROVED PROMPT

Role: senior Python engineer. Context: production ETL job on OCI, input file may exceed 2GB. Task: write a function that streams a CSV from OCI Object Storage and yields rows as dicts without loading the full file into memory. Constraints: use oci-python-sdk, include type hints and docstring, handle missing files with a clear exception.

EXPECTED OUTPUT

```
def stream_csv_from_object_storage(bucket: str, object_name:
str) -> Iterator[dict]:
    """Streams CSV rows..."""
    ...
```

WHY IT'S BETTER

Naming the runtime environment, scale constraint, and library constraints prevents a naive `pandas.read_csv()` answer that would fail on a 2GB file in a memory-constrained job.

Log Analysis

POOR PROMPT

What's wrong with these logs?

IMPROVED PROMPT

Role: site reliability engineer. Task: analyze the attached 500-line application log and identify the 3 most likely root causes of the recurring 'Connection reset' errors, ranked by evidence strength. Output format: a ranked list, each with the supporting log-line timestamps cited.

EXPECTED OUTPUT

```
1. Connection pool exhaustion (evidence: 14 'timeout waiting
for connection' lines at 09:14-09:22) 2. Downstream service
restart (evidence: correlated 502s at 09:15) 3. Network blip
(weaker evidence, isolated occurrence at 03:02)
```

WHY IT'S BETTER

Asking for ranked, evidence-cited causes (not a single guess) produces a triage-ready analysis an engineer can act on immediately.

Error Troubleshooting

POOR PROMPT

Fix ORA-01555.

IMPROVED PROMPT

Role: Oracle performance DBA. Context: ORA-01555 'snapshot too old' occurring on a 45-minute batch job against an Autonomous Database, UNDO_RETENTION=900. Task: list the 3 most probable causes for this specific context and the diagnostic query to confirm each, before suggesting a fix. Constraints: do not suggest disabling UNDO management.

EXPECTED OUTPUT

1. UNDO_RETENTION too low for a 45-min job – confirm via V\$UNDOSTAT... 2. Long-running query fetching against actively-updated rows... 3. Undo tablespace undersized...

WHY IT'S BETTER

Constraining the context (Autonomous DB, specific retention value) and forbidding a lazy non-fix (disabling undo) keeps the answer specific and production-safe.

Documentation Generation

POOR PROMPT

Write docs for this API.

IMPROVED PROMPT

Role: technical writer for external developer docs. Task: generate OpenAPI-style documentation for the endpoint below: purpose, request/response schema, 2 example calls (success and error), and required IAM policy. Output format: Markdown matching our existing docs style (see example section heading pattern).

EXPECTED OUTPUT

```
## POST /v1/invoices/reconcile
**Purpose:** Triggers automated reconciliation...
**Request:** { "invoice_id": string }
**Response 200:** { "status": "reconciled" }...
```

WHY IT'S BETTER

Specifying the doc style, required sections, and an error example produces publish-ready documentation instead of a generic paragraph description.

Test-Case Generation

POOR PROMPT

Write tests for this function.

IMPROVED PROMPT

Role: QA engineer practicing boundary-value analysis. Task: generate pytest unit tests for the discount-calculation function below, covering: normal case, zero quantity, negative quantity (should raise), maximum discount cap, and floating-point rounding edge cases. Output format: a single pytest file using parametrize.

EXPECTED OUTPUT

```
@pytest.mark.parametrize("qty,price,expected", [(1,100,100),
(0,100,0),(1000,100,8000)])
def test_discount_normal(qty, price, expected): ...
def test_negative_quantity_raises(): ...
```

WHY IT'S BETTER

Naming specific edge cases (not just 'write tests') ensures boundary and error conditions are covered, not only the happy path.

Database Optimization Analysis

POOR PROMPT

Make this query faster.

IMPROVED PROMPT

Role: Oracle performance tuning specialist. Context: attached is the EXPLAIN PLAN and AWR SQL Monitor report for a query taking 40 seconds on a 200M-row partitioned table. Task: identify the top 2 performance bottlenecks from the plan and recommend specific fixes (indexing, partition pruning, or rewrite). Constraints: no schema changes requiring downtime.

EXPECTED OUTPUT

1. Full table scan on ORDERS despite date filter – partition pruning not occurring because date column is wrapped in TO_CHAR(). Fix: filter on raw DATE column. 2. Nested loop join on 2M-row set – add index on FK column...

WHY IT'S BETTER

Supplying the actual execution plan and constraining to zero-downtime fixes produces a specific, deployable recommendation instead of generic tuning advice.

RAG Question Answering

POOR PROMPT

What's our return policy?

IMPROVED PROMPT

Role: customer support assistant. Context: use ONLY the following retrieved policy excerpts — do not use outside knowledge. Task: answer the question, citing the section number. If the excerpts don't contain the answer, respond exactly: "I don't have that information in our current policy documents."
[retrieved excerpts below] Question: What's our return policy for custom-machined parts?

EXPECTED OUTPUT

Per Section 7.3, custom-machined parts are non-returnable unless the defect is a manufacturing error verified within 15 days of delivery.

WHY IT'S BETTER

Explicit grounding instructions and a defined fallback response prevent hallucinated policy details — critical for a support assistant that must never invent policy.

MODULE

11

Agentic AI

Learning Objectives

- Differentiate a chatbot, an AI assistant, and an AI agent
- Name every component of agent architecture: goal, planning, memory, tools
- Trace the ReAct loop and explain function/tool calling
- Design a multi-step enterprise incident-response agent

What Is Agentic AI?

OBJECTIVE *Define Agentic AI as autonomous, multi-step, tool-using execution.*

- Agentic AI uses an LLM as a reasoning engine inside a loop that plans a sequence of actions, executes them using external tools, observes the results, and continues until a goal is achieved — with limited or no step-by-step human direction.
- This is a qualitative shift from a single prompt-response exchange: an agent decides not just what to say, but what to do next, potentially across many tool calls and minutes of autonomous operation.
- Agentic AI is built on Generative AI (Module 9) but adds three things a plain LLM call lacks: persistent goal-directed planning, the ability to invoke external tools/systems, and memory of its own intermediate steps.
- Enterprise agents are typically scoped narrowly and wrapped in guardrails (Module 1's rule-based-AI pattern reappears here) — full unconstrained autonomy is rare and generally undesirable in production.

Chatbot vs. AI Assistant vs. AI Agent

Dimension	Chatbot	AI Assistant	AI Agent
Primary behavior	Responds to messages in a conversation	Answers questions and performs simple bounded actions	Plans and executes multi-step tasks autonomously
Tool use	None, or very limited	Often integrates 1-2 tools/APIs directly	Dynamically selects and chains multiple tools
Autonomy	None — one turn at a time, human-driven	Low — follows explicit user requests	High — pursues a goal across many steps with minimal input
Memory	Session-only, if any	Session or user-profile memory	Task memory across the full multi-step execution, sometimes long-term
Example	FAQ bot answering from a script	“Schedule this meeting for me”	“Diagnose and remediate this production incident” end-to-end

Agent Architecture: Goal, Planning, Reasoning

OBJECTIVE *Define the components that let an agent decide what to do.*

- **Goal:**

The objective the agent is working toward, provided by the user or a triggering system event (e.g., “resolve this P2 incident”).

- **Planning:**

Decomposing the goal into an ordered sequence of intermediate steps or sub-goals before (or while) acting — e.g., “read the logs, then query the database, then propose a fix.”

- **Reasoning:**

The LLM's step-by-step inference about what the current situation means and what to do next — informed by chain-of-thought-style prompting (Module 10) internally, even when not shown to the user.

Agent Architecture: Tools, Memory, and the Feedback Loop

OBJECTIVE *Define the components that let an agent act, remember, and self-correct.*

- **Tool usage:**

Invoking external functions, APIs, or systems (a database query, a REST call, a script) to gather information or take action the LLM cannot do by itself.

- **Memory:**

Short-term memory of the current task's steps/results, and optionally long-term memory persisted across sessions (e.g., stored in a vector store or database).

- **Observation:**

The result returned by a tool call or action — new information the agent incorporates into its next reasoning step.

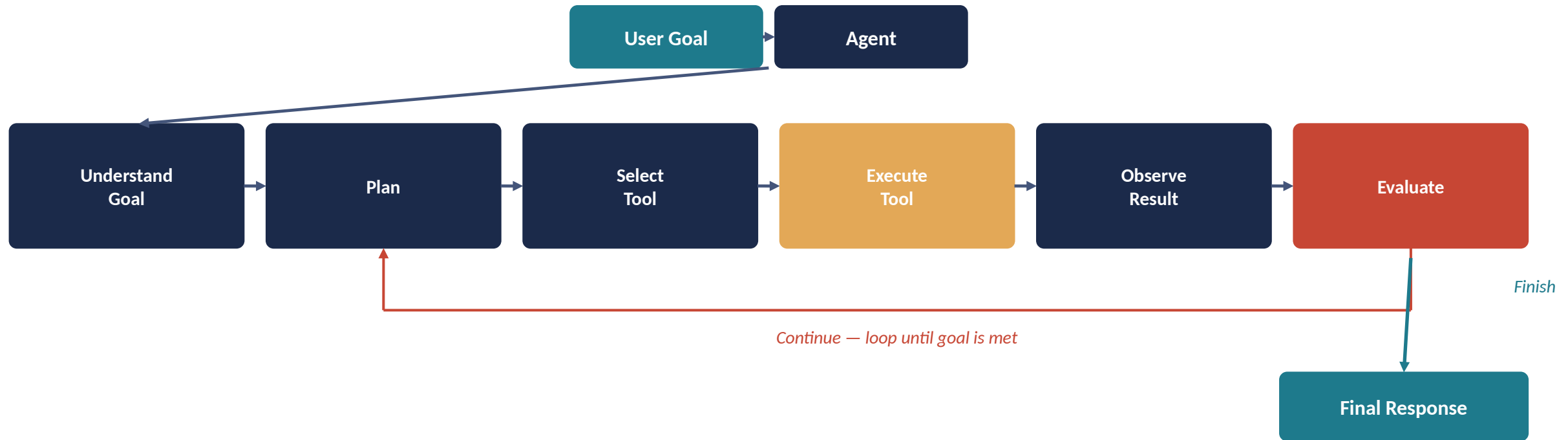
- **Action:**

The concrete step the agent takes: a tool call, a message, or a final answer.

- **Feedback and Reflection:**

The agent evaluates whether its last action moved it closer to the goal, and self-corrects course if not — critical for reliability in longer-running tasks.

Agentic AI Workflow



The agent loops through Plan → Select Tool → Execute → Observe → Evaluate until the goal is reached, then returns a final response.

Reading the Agentic AI Workflow

- 1. User Goal → Agent:** the objective is handed to the agent, which begins the loop by understanding what is actually being asked.
- 2. Plan:** the agent decomposes the goal into an ordered sequence of steps.
- 3. Select Tool → Execute Tool:** for the current step, the agent chooses the most relevant available tool and invokes it against a real system.
- 4. Observe Result:** the tool's output is captured as new information.
- 5. Evaluate:** the agent checks whether the goal has been met.
- 6. Continue or Finish:** if not met, the loop returns to Plan with the new information; if met, the agent exits the loop and returns the Final Response.

The ReAct Pattern

OBJECTIVE *Understand the prompting pattern that operationalizes the agent loop.*

- ReAct (Reason + Act) interleaves explicit reasoning steps with tool-calling actions in a single running transcript, rather than reasoning silently then acting once.
- **Pattern: Thought → Action → Observation → Thought → Action ... repeated until a Final Answer.**
- Each “Thought” is the model reasoning in natural language about what it has learned so far and what to try next; each “Action” is a tool call; each “Observation” is that tool's result fed back into the transcript.
- This interleaving lets the agent adapt its plan based on real intermediate results, rather than committing to a rigid upfront plan that may fail partway through — the core mechanism behind most production agent frameworks, including OCI Generative AI Agents (Module 12).

ENTERPRISE EXAMPLE

Thought: “I need the customer's order history.” Action: call `get_orders(customer_id)`. Observation: [3 orders returned]. Thought: “Order #3 is overdue — checking shipment status next.”

Function / Tool Calling

OBJECTIVE *Understand the mechanism that lets an LLM take real action.*

- Function (tool) calling lets a developer describe available functions to the model — name, description, and a JSON-schema parameter signature — and the model responds with a structured request to call one, with arguments it has inferred from the conversation.
- The application (not the model) actually executes the function against real systems, then returns the result to the model as an observation for its next reasoning step — the model never has direct system access.
- This is the concrete mechanism underlying “tool usage” in the agent loop, and how OCI Generative AI's Function Calling Tool and Generative AI Agents' SQL/RAG/API tools operate.
- Well-designed tool schemas — clear names, tight parameter types, explicit descriptions — measurably improve an agent's tool-selection accuracy.

Multi-Agent Architecture

OBJECTIVE *Understand when and why to decompose one agent into several.*

- A multi-agent system splits responsibility across several specialized agents rather than one generalist agent handling everything — e.g., a Planner agent, a Database agent, a Code-review agent, and a Reporting agent, coordinated by an orchestrator.
- Benefits: each agent gets a narrower, more reliable tool set and prompt; failures are easier to isolate; specialized agents can be reused across workflows.
- Coordination patterns: a central orchestrator agent delegating sub-tasks (hierarchical), or peer agents passing messages directly (collaborative).
- OCI Generative AI Agents' Agent Tool explicitly supports orchestrating a network of specialized agents collaborating on a task — the managed-service equivalent of this pattern.

Agentic Incident Response — End to End

An on-call engineering copilot handles a production alert autonomously:

- 1. Receives an incident:** A P2 alert fires: “Elevated error rate on the payments API.”
- 2. Reads logs:** The agent calls a log-query tool to pull the last 15 minutes of application logs for the affected service.
- 3. Queries a database:** It queries the operations database for recent deployments and configuration changes correlated with the alert window.
- 4. Analyzes the issue:** It reasons over the logs and change history to identify the most probable root cause (e.g., a recent connection-pool config change).
- 5. Creates a remediation recommendation:** It drafts a specific proposed fix (e.g., revert the connection-pool size change) with supporting evidence.
- 6. Calls an approved automation tool:** Within a pre-authorized guardrail, it triggers a rollback runbook via an approved OCI Functions tool — or, for higher-risk actions, requests human approval first.
- 7. Reports the result:** It posts a structured incident summary — cause, action taken, verification status — to the on-call channel.

A Minimal Tool-Calling Agent Loop

```
tools = [{
    "name": "query_incident_db",
    "description": "Look up recent deployments for a service",
    "parameters": {"type": "object",
        "properties": {"service": {"type": "string"}}}
}]

response = chat_client.chat(
    messages=messages, tools=tools, tool_choice="auto")

while response.tool_calls:
    for call in response.tool_calls:
        result = execute_tool(call.name, call.arguments) # real system call
        messages.append({"role": "tool", "name": call.name,
            "content": result}) # observation
        response = chat_client.chat(messages=messages, tools=tools)

print(response.content) # final grounded answer
```

EXPECTED OUTPUT

Root cause: connection_pool_size reduced from 200 to 20
in deploy #4471 at 09:12. Recommended action: revert.

ENTERPRISE USE CASE

Core loop pattern behind OCI Generative AI Agents' Function
Calling and Agent tools.

Traditional Software vs. ML vs. GenAI vs. Agentic AI

Dimension	Traditional Software	Machine Learning	Generative AI	Agentic AI
Behavior source	Explicit hand-written logic	Patterns learned from data	Patterns learned from massive text/data corpora	LLM reasoning + planning + tools
Adapts to new input how	Doesn't, without a code change	Generalizes within its trained distribution	Generalizes broadly via pretraining	Plans a new action sequence per situation
Output	Deterministic result	A prediction	Generated content	A completed multi-step task
Autonomy	None	None (a component in a pipeline)	Low (single response per call)	High (multi-step, tool-using, self-correcting)

MODULE

12

Oracle Cloud Infrastructure (OCI)

Learning Objectives

- Map every AI/ML/DL/GenAI concept from Modules 1–11 onto a named OCI service
- Explain the purpose, AI use case, and example architecture for 13 OCI services
- Distinguish managed AI services from build-your-own ML infrastructure on OCI
- Compare on-premise AI operations to OCI's managed AI stack

OCI's AI Stack, at a Glance

OBJECTIVE *See how OCI services map to the AI hierarchy from Module 1.*

- OCI offers three layers of AI capability, from least to most build effort: pretrained AI Services (call an API, no data science required), OCI Generative AI (call or lightly fine-tune a foundation LLM), and OCI Data Science (build, train, and deploy fully custom models).
- Every AI workload also depends on OCI's foundational platform services — Object Storage for data, Autonomous Database for transactional + vector data, Compute/OKE for custom hosting, and IAM/Logging/Monitoring/API Gateway for security and operations.
- The next 13 slides cover one service each: Purpose, AI use case, and example architecture — building the vocabulary Module 13 assembles into complete reference architectures.

OCI Generative AI

OBJECTIVE *Managed Foundation Models*

- **Purpose:**

A fully managed service providing on-demand access to hosted foundation LLMs (Cohere, Meta Llama family) for chat, text generation, embeddings, and reranking — plus dedicated AI clusters for fine-tuning and hosting custom models.

- **AI Use Case:**

Powering enterprise chat assistants, RAG applications, and agentic workflows (via the OCI Responses API with Function Calling, File Search, and MCP tool support) without hosting your own model infrastructure.

ENTERPRISE EXAMPLE

Example architecture: Application → OCI Generative AI Chat/Embeddings API → hosted foundation model → grounded response, optionally combined with Oracle DB 23ai Vector Search for RAG.

OCI AI Services

OBJECTIVE *Pretrained Vision, Language, Speech APIs*

- **Purpose:**

A family of prebuilt, pretrained AI APIs — Language (NLU, sentiment, entity extraction, translation), Vision (image analysis, OCR), Speech (transcription/synthesis), and Document Understanding (structured document extraction) — usable with zero model training.

- **AI Use Case:**

Extracting text and key fields from scanned invoices; classifying support-ticket sentiment; transcribing customer calls for downstream analysis.

ENTERPRISE EXAMPLE

Example architecture: Document/image lands in Object Storage → OCI AI Services API call → structured JSON output → written to Autonomous Database for the business application.

OCI Data Science

OBJECTIVE *Custom Model Build & Deploy Platform*

- **Purpose:**

A managed platform for the full custom ML lifecycle: JupyterLab-based Notebook Sessions, a Model Catalog for versioning, Jobs for scheduled/scaled training runs, Model Deployment for hosting REST inference endpoints, and AI Quick Actions for deploying open-source/foundation models with minimal setup.

- **AI Use Case:**

Training and deploying a bespoke defect-detection CNN (Module 5) or a custom demand-forecasting LSTM (Module 7) that off-the-shelf AI Services cannot provide.

ENTERPRISE EXAMPLE

Example architecture: Training data in Object Storage → Notebook Session/Job trains the model → registered in Model Catalog → Model Deployment exposes a REST endpoint → application calls it for inference.

OCI Object Storage

OBJECTIVE *Durable Storage for Unstructured Data*

- **Purpose:**

Highly durable, scalable, S3-compatible object storage — the default landing zone for documents, images, model artifacts, and training datasets across every AI workload on OCI.

- **AI Use Case:**

Central repository for raw scanned documents awaiting OCR, training datasets for OCI Data Science, and RAG source documents prior to chunking/embedding.

ENTERPRISE EXAMPLE

Example architecture: Documents uploaded to an Object Storage bucket → an OCI Event fires on new-object creation → triggers an OCI Function that starts the AI processing pipeline.

OCI Autonomous Database

OBJECTIVE *Self-Driving Oracle Database*

- **Purpose:**

A fully managed Oracle Database (Autonomous Data Warehouse / Autonomous Transaction Processing) that automates tuning, patching, backup, and scaling using Oracle's own embedded ML — the “self-driving, self-securing, self-repairing” database referenced in Module 1.

- **AI Use Case:**

Hosting both transactional application data and — via Oracle Database 23ai's AI Vector Search — the vector embeddings for RAG, without operating a separate specialized vector database.

ENTERPRISE EXAMPLE

Example architecture: Application and RAG pipeline both connect to one Autonomous Database instance; automated tuning maintains query SLAs as vector-search and transactional load both grow.

Oracle Database AI Vector Search

OBJECTIVE *Native Vector Search in Oracle DB 23ai*

- **Purpose:**

A native VECTOR data type plus vector indexing (HNSW, IVF) and SQL operators (e.g., VECTOR_DISTANCE) built directly into Oracle Database 23ai, enabling semantic similarity search alongside ordinary relational queries and transactions.

- **AI Use Case:**

The retrieval layer of an enterprise RAG system (Module 9) — one SQL query can combine vector similarity search with relational filters (e.g., “similar AND status='ACTIVE'”) in a single transactionally-consistent step.

ENTERPRISE EXAMPLE

Example architecture: Document chunks embedded and stored in a VECTOR column → query embedded at request time → SQL SELECT ... ORDER BY VECTOR_DISTANCE(...) → top-k rows returned with full relational context attached.

OCI Functions

OBJECTIVE *Serverless, Event-Driven Compute*

- **Purpose:**

A serverless, container-based Functions-as-a-Service platform (built on the open-source Fn Project) that runs code on demand in response to events, with no server management and pay-per-invocation billing.

- **AI Use Case:**

The glue logic behind agent tool calls (Module 11) and event-driven document-processing pipelines — e.g., a function invoked whenever a new invoice lands in Object Storage.

ENTERPRISE EXAMPLE

Example architecture: OCI Event (new object in bucket) → triggers Function → Function calls an AI Service or Data Science model endpoint → writes structured results to Autonomous Database.

API Gateway

OBJECTIVE *Managed API Front Door*

- **Purpose:**

A managed entry point for publishing APIs backed by Functions, Compute, or other OCI services — handling authentication, request/response transformation, rate limiting, and routing in front of your backend.

- **AI Use Case:**

Securely exposing a RAG chatbot or agent backend to external or partner-facing client applications with consistent authentication and throttling.

ENTERPRISE EXAMPLE

Example architecture: Client application → API Gateway (auth + rate limit + routing) → backend service (Functions or Compute running the RAG/agent application) → OCI Generative AI.

IAM (Identity and Access Management)

OBJECTIVE *Least-Privilege Access Control*

- **Purpose:**

Policies, groups, dynamic groups, and compartments that control precisely which principals — human users or OCI resources like Functions — can perform which actions on which resources.

- **AI Use Case:**

Restricting an agent's Function identity to only the specific Generative AI models, Object Storage buckets, and database credentials it needs — no broader access, limiting the blast radius of a misbehaving agent.

ENTERPRISE EXAMPLE

Example architecture: A dynamic group matching the agent's Function OCID → an IAM policy grants “use generative-ai-family” and read access to one named bucket, scoped to one compartment — enforced on every API call.

Logging

OBJECTIVE *Centralized Log Aggregation*

- **Purpose:**

A managed service that aggregates logs from OCI resources and custom application/service logs into a single, searchable, exportable destination.

- **AI Use Case:**

Capturing every Generative AI request/response and every agent tool-call trace for audit, debugging, and compliance review.

ENTERPRISE EXAMPLE

Example architecture: Application and Functions emit custom logs → OCI Logging aggregates them → searched directly or exported to Object Storage / a SIEM for long-term retention and analysis.

Monitoring

OBJECTIVE *Metrics, Alarms, and Notifications*

- **Purpose:**

Collects metrics from OCI resources and custom application metrics, and triggers alarms/notifications when defined thresholds are crossed.

- **AI Use Case:**

Alerting the on-call team when Generative AI inference latency spikes, error rates rise, or an agent's tool-call failure rate crosses an acceptable threshold.

ENTERPRISE EXAMPLE

Example architecture: Service/application emits metrics → Monitoring evaluates alarm rules → breach triggers Notifications (ONS) → delivered to the on-call Slack/email/PagerDuty-equivalent channel.

Compute

OBJECTIVE *VM and Bare-Metal Instances*

- **Purpose:**

Flexible VM and bare-metal compute shapes, including GPU shapes, for workloads that need infrastructure control beyond what managed services provide.

- **AI Use Case:**

Self-hosting a custom or open-source model server, or running large-scale GPU training jobs outside OCI Data Science's managed environment.

ENTERPRISE EXAMPLE

Example architecture: A GPU Compute instance runs a containerized inference server behind a Load Balancer, called by the application exactly like a managed AI service endpoint.

Kubernetes / OKE

OBJECTIVE *Managed Container Orchestration*

- **Purpose:**

Oracle Container Engine for Kubernetes (OKE) — a managed Kubernetes service for deploying, scaling, and operating containerized applications, including multi-component AI systems.

- **AI Use Case:**

Orchestrating a multi-service agentic application — an orchestrator, a retrieval service, and several tool microservices — as independently scalable containers.

ENTERPRISE EXAMPLE

Example architecture: An OKE cluster runs the RAG/agent microservices as pods, auto-scaling with load, integrated with OCI Logging and Monitoring for full observability.

On-Premise AI vs. OCI AI

Dimension	On-Premise AI	OCI AI
Infrastructure	Self-procured, self-managed GPU/CPU hardware	Managed Compute, Data Science, and Generative AI — provisioned on demand
Model access	Must self-host every model, including foundation LLMs	Pay-per-use access to hosted foundation models via OCI Generative AI
Scaling	Manual capacity planning and procurement lead time	Elastic scaling of Compute, OKE, and managed AI services
Operational burden	Full ownership of patching, tuning, HA, DR	Autonomous Database and managed services absorb most operational burden
Data gravity	AI compute must be brought to on-prem data, or data exported	Oracle Database 23ai Vector Search keeps AI retrieval co-located with existing Oracle enterprise data
Time to first result	Weeks to months (procurement, setup, MLOps tooling)	Hours to days for pretrained/Generative AI services; days to weeks for custom models

MODULE

13

OCI Enterprise AI Architecture

Learning Objectives

- Assemble Module 12's OCI services into three complete reference architectures
- Design an Enterprise RAG pipeline on Oracle Database 23ai
- Design a multi-tool Agentic AI architecture on OCI
- Design an Enterprise Document Intelligence pipeline

From Services to Architectures

OBJECTIVE *Preview the three reference architectures this module assembles.*

- Module 12 introduced 13 individual OCI services. Real enterprise systems compose several of them into one end-to-end architecture — this module walks three of the most common patterns in full detail.

- **Use Case 1 — Enterprise RAG:**

grounding an LLM in enterprise knowledge using Oracle Database 23ai Vector Search.

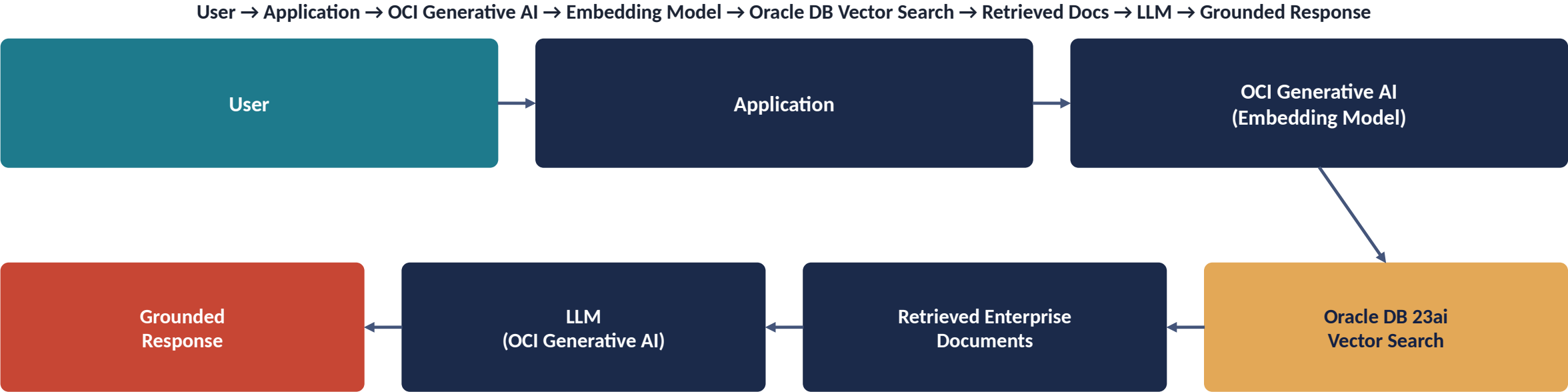
- **Use Case 2 — Agentic AI on OCI:**

an autonomous agent selecting among multiple enterprise tools to complete a task.

- **Use Case 3 — Enterprise Document Intelligence:**

turning unstructured documents into structured, actionable business data.

Enterprise RAG Architecture

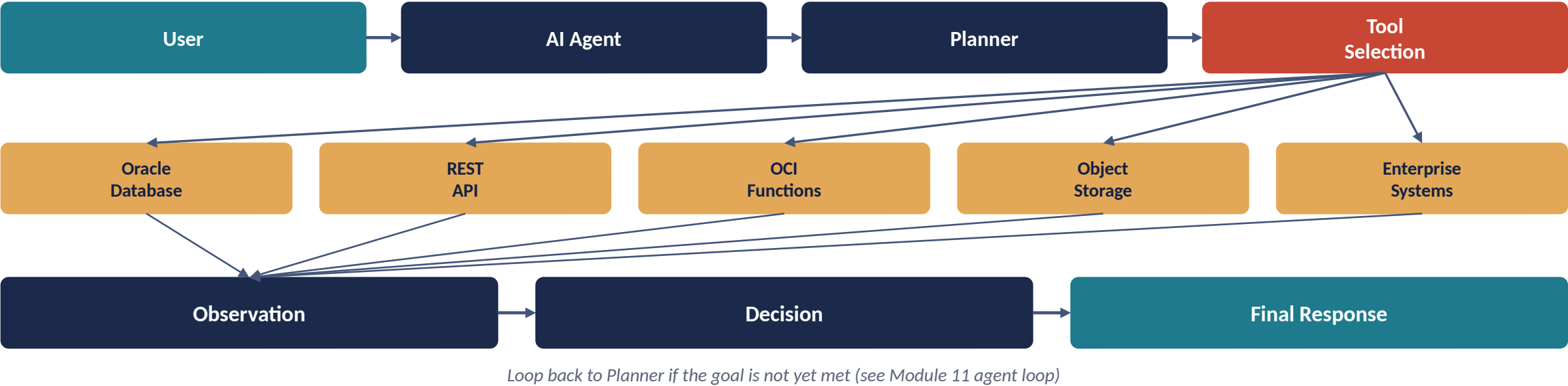


A grounded, citable response is produced by combining OCI Generative AI with Oracle Database 23ai's native vector search over enterprise documents.

Enterprise RAG — Step by Step

- 1. User submits a question:** via the application's chat interface.
- 2. Application calls OCI Generative AI's embedding model:** converting the question into a query vector.
- 3. Oracle Database 23ai Vector Search:** finds the nearest document-chunk embeddings using VECTOR_DISTANCE, in the same database as the enterprise's operational data.
- 4. Retrieved documents:** the top-k matching passages are returned with their source text and metadata.
- 5. LLM generation:** OCI Generative AI's chat model generates an answer grounded strictly in the retrieved passages.
- 6. Grounded response:** returned to the user with citations back to the source documents, satisfying the grounding discipline from Module 10.

Agentic AI Architecture on OCI

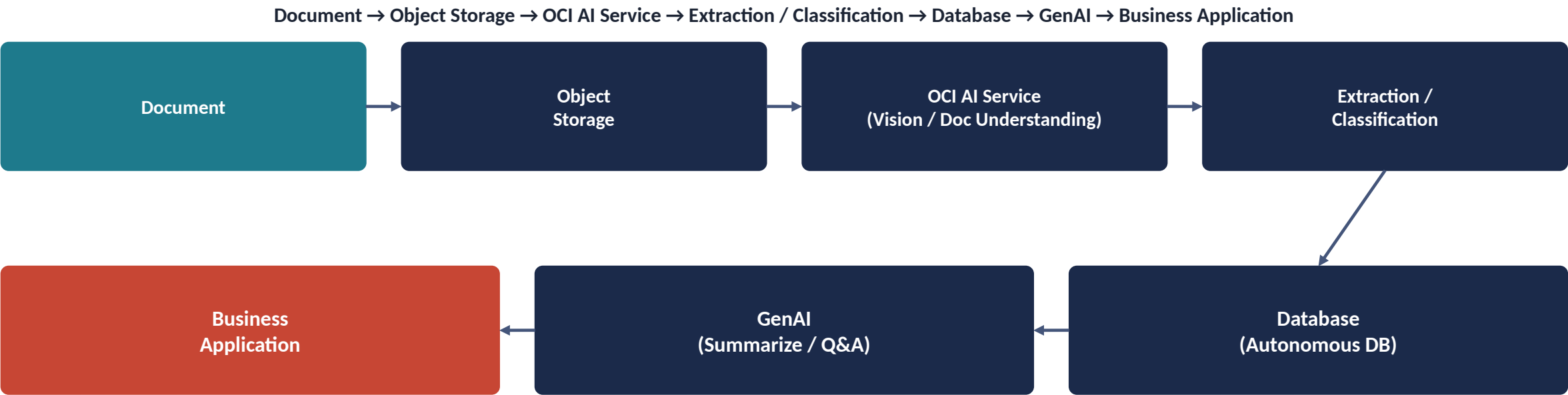


The agent's planner selects among several enterprise tools per step, observes each result, and decides whether to continue or respond.

Agentic AI on OCI — Step by Step

1. **User goal:** enters the AI Agent, built on OCI Generative AI Agents or a custom orchestration layer.
2. **Planner:** decomposes the goal and selects the next best action.
3. **Tool selection:** the agent chooses from its available enterprise tools — Oracle Database (SQL Tool), a REST API, OCI Functions, Object Storage, or other enterprise systems — based on what the current step requires.
4. **Execution:** the selected tool runs against the real system, governed by the IAM policy scoped to that tool (Module 12).
5. **Observation:** the tool's result is captured and fed back to the planner.
6. **Decision:** the agent evaluates whether the goal is met; if not, it loops back to the planner for another tool-selection cycle; if met, it returns the Final Response.

Enterprise Document Intelligence Architecture



Unstructured documents become structured, queryable, and GenAI-summarized business data.

Document Intelligence — Step by Step

- 1. Document ingested:** a scanned invoice, contract, or form is uploaded to an Object Storage bucket.
- 2. OCI AI Service processes it:** Document Understanding or Vision extracts text, tables, key-value pairs, and classifies the document type.
- 3. Extraction / classification:** structured fields (vendor, amount, due date) are produced from unstructured input.
- 4. Database:** extracted structured data is written to Autonomous Database, joining the enterprise's existing operational data.
- 5. GenAI layer:** OCI Generative AI summarizes, flags anomalies, or answers natural-language questions over the now-structured document data.
- 6. Business application:** surfaces the summary, extracted fields, or Q&A answer to the end user — e.g., an AP clerk reviewing a queued invoice.

Choosing the Right Architecture Pattern

Pattern	Use When	Core OCI Services	Key Design Concern
Enterprise RAG	Users need grounded answers from enterprise knowledge	OCI Generative AI, Oracle DB 23ai Vector Search	Chunking strategy and retrieval relevance
Agentic AI	A multi-step task must be executed, not just answered	OCI Generative AI Agents, Functions, IAM	Tool-permission scoping and human-approval gates
Document Intelligence	Unstructured documents must become structured data	OCI AI Services, Object Storage, Autonomous Database	Extraction accuracy and downstream data validation

Oracle Enterprise Use Cases

Learning Objectives

- See 12 complete, production-shaped Oracle/OCI AI use cases
- Connect business problem to architecture to concrete OCI services
- Read example prompts and expected results for each use case
- Identify the security consideration every use case must address

Oracle Database Support Assistant

- **Business Problem:**

DBAs spend hours searching documentation and MOS notes to diagnose Oracle errors.

- **AI Solution:**

A RAG chatbot grounded in Oracle documentation, internal runbooks, and past incident tickets.

- **Architecture:**

User → chat app → OCI Generative AI → Oracle DB 23ai Vector Search over docs/runbooks → grounded answer.

- **OCI Services:**

OCI Generative AI, Oracle DB 23ai Vector Search, Object Storage, API Gateway.

EXAMPLE PROMPT

```
"Using only the attached runbook excerpts,
explain the
likely cause of ORA-01555 on our 45-minute
nightly
batch job and the recommended fix."
```

EXPECTED RESULT

A grounded explanation citing the specific runbook section, plus the diagnostic query to confirm the root cause before applying a fix.

SECURITY CONSIDERATIONS

Restrict retrieval corpus to approved internal documentation; log every query/response pair for audit; never let the assistant execute DDL/DML directly.

SQL / PL/SQL Coding Assistant

- **Business Problem:**

Developers lose time writing boilerplate SQL/PLSQL and debugging syntax across large schemas.

- **AI Solution:**

An LLM-based coding assistant with schema-aware prompting for SQL generation, explanation, and review.

- **Architecture:**

IDE plugin / internal portal → API Gateway → OCI Generative AI (schema context injected) → generated/reviewed code.

- **OCI Services:**

OCI Generative AI, API Gateway, IAM.

EXAMPLE PROMPT

```
"Given table ORDERS(order_id, customer_id,
order_date,
status, total_amount), write an Oracle SQL
query for
monthly revenue in 2025, COMPLETED orders
only."
```

EXPECTED RESULT

Valid, dialect-correct Oracle SQL matching the supplied schema, ready to review and run.

SECURITY CONSIDERATIONS

Never grant the assistant direct database execution rights; generated SQL is reviewed/executed by the developer through normal change control.

Enterprise RAG Chatbot

- **Business Problem:**

Employees can't quickly find answers scattered across HR policies, IT wikis, and product docs.

- **AI Solution:**

A general-purpose enterprise RAG chatbot grounded across multiple internal knowledge sources.

- **Architecture:**

User → chat UI → OCI Generative AI Agents (RAG Tool) → Oracle DB 23ai Vector Search → grounded, cited answer.

- **OCI Services:**

OCI Generative AI Agents, Oracle DB 23ai Vector Search, Object Storage, IAM.

EXAMPLE PROMPT

```
"What is the process for requesting expensed travel over $2,000, and who needs to approve it?"
```

EXPECTED RESULT

A cited answer pulled from the current travel & expense policy document, with a link back to the source section.

SECURITY CONSIDERATIONS

Apply document-level access control so retrieval only surfaces content the requesting user is authorized to see.

IT Operations Assistant

- **Business Problem:**

IT support staff manually triage repetitive tickets (password resets, access requests, VPN issues).

- **AI Solution:**

A conversational assistant that answers common IT questions and triggers simple approved actions.

- **Architecture:**

Employee chat → OCI Generative AI Agents → Function Calling Tool → ITSM/IAM system → confirmation.

- **OCI Services:**

OCI Generative AI Agents, OCI Functions, IAM, Logging.

EXAMPLE PROMPT

```
"I'm locked out of my VPN account, can you  
reset  
my access?"
```

EXPECTED RESULT

The assistant verifies identity, calls the approved reset-access function, and confirms completion with a ticket reference.

SECURITY CONSIDERATIONS

Tool calls must re-verify caller identity independent of the chat session; every automated action is logged and reversible.

Incident Management Agent

- **Business Problem:**

On-call engineers spend the first 15–30 minutes of an incident just gathering diagnostic data.

- **AI Solution:**

An agentic AI system that reads logs, queries operational data, and proposes remediation automatically (Module 11).

- **Architecture:**

Alert → AI Agent → log/database tools → root-cause analysis → approved automation tool → incident report.

- **OCI Services:**

OCI Generative AI Agents, OCI Functions, Logging, Monitoring, IAM.

EXAMPLE PROMPT

```
"A P2 alert fired for elevated error rates on the payments API. Diagnose the likely root cause."
```

EXPECTED RESULT

A ranked root-cause analysis citing specific log lines and recent deployments, with a proposed remediation action.

SECURITY CONSIDERATIONS

High-impact remediation actions require human approval before execution; the agent's tool scope is limited to read + pre-approved runbooks only.

Customer Support Agent

- **Business Problem:**

Support teams handle high ticket volume with inconsistent response quality and slow resolution times.

- **AI Solution:**

A RAG-grounded customer support agent that answers from product docs and can escalate or take approved actions.

- **Architecture:**

Customer chat → OCI Generative AI Agents → RAG Tool (product docs) + API Tool (order system) → grounded response or escalation.

- **OCI Services:**

OCI Generative AI Agents, Oracle DB 23ai Vector Search, API Gateway.

EXAMPLE PROMPT

"Where is my order #48213 and can I change the delivery address?"

EXPECTED RESULT

Live order status retrieved via the API tool, with an address-change action taken if within the allowed policy window, else escalated to a human agent.

SECURITY CONSIDERATIONS

PII (order/customer data) is only ever fetched live, never embedded into the vector store; escalation path exists for anything outside policy.

Document Processing

▪ Business Problem:

AP teams manually key data from hundreds of scanned invoices daily, causing delay and data-entry error.

▪ AI Solution:

Automated document intelligence pipeline extracting structured invoice data for straight-through processing (Module 13, Use Case 3).

▪ Architecture:

Scanned invoice → Object Storage → OCI Document Understanding → extracted fields → Autonomous Database → AP system.

▪ OCI Services:

OCI AI Services (Document Understanding), Object Storage, OCI Functions, Autonomous Database.

EXAMPLE PROMPT

```
(No LLM prompt – OCI AI Service API call)
extract:
vendor, invoice_number, amount, due_date,
line_items
```

EXPECTED RESULT

Structured JSON with extracted fields and confidence scores, auto-posted to the AP queue for exceptions-only human review.

SECURITY CONSIDERATIONS

Low-confidence extractions are routed to human review rather than auto-posted; extracted financial data is access-controlled like any AP record.

Log Analysis

- **Business Problem:**

Terabytes of application/system logs make manual anomaly spotting infeasible at enterprise scale.

- **AI Solution:**

An LSTM- or LLM-based log analysis system that flags anomalous sequences and summarizes likely causes (Modules 6–7).

- **Architecture:**

Log stream → OCI Logging → anomaly-detection model (Data Science) or LLM summarization → alert with explanation.

- **OCI Services:**

OCI Logging, OCI Data Science, OCI Generative AI, Monitoring.

EXAMPLE PROMPT

```
"Summarize the 3 most likely root causes of the
recurring Connection Reset errors in the
attached
log excerpt, citing supporting timestamps."
```

EXPECTED RESULT

A ranked, evidence-cited summary handed to the on-call engineer, cutting initial triage time significantly.

SECURITY CONSIDERATIONS

Logs may contain sensitive request data — apply field-level redaction before any content leaves the logging boundary for LLM processing.

Predictive Maintenance

- **Business Problem:**

Unplanned equipment failures on the shop floor cause costly downtime and emergency repairs.

- **AI Solution:**

An LSTM-based model forecasting equipment failure risk from historical sensor telemetry (Module 7).

- **Architecture:**

Sensor stream → Object Storage / Streaming → OCI Data Science model → risk score → Autonomous Database → maintenance dashboard.

- **OCI Services:**

OCI Data Science, Object Storage, Autonomous Database, Monitoring.

EXAMPLE PROMPT

```
(Model inference, not a chat prompt) input: 30-day  
rolling vibration + temperature window for  
asset #118
```

EXPECTED RESULT

A failure-risk score and recommended maintenance window, surfaced days ahead of a predicted failure.

SECURITY CONSIDERATIONS

Model inputs/outputs tied to physical assets should follow the same access controls as the underlying operational technology (OT) systems.

Knowledge-Management Assistant

▪ Business Problem:

Institutional knowledge is siloed in wikis, tickets, and individual experts' heads, and is lost with staff turnover.

▪ AI Solution:

A RAG assistant continuously indexing internal knowledge sources with source-cited answers.

▪ Architecture:

Knowledge sources → ingestion pipeline (chunk + embed) → Oracle DB 23ai Vector Search → OCI Generative AI → cited answer.

▪ OCI Services:

OCI Generative AI, Oracle DB 23ai Vector Search, Object Storage, OCI Functions (ingestion pipeline).

EXAMPLE PROMPT

```
"What is our documented process for onboarding a new manufacturing supplier?"
```

EXPECTED RESULT

A synthesized, cited answer drawn from the current onboarding wiki page, refreshed automatically as source documents change.

SECURITY CONSIDERATIONS

Ingestion pipeline must respect source-document access permissions when indexing, so retrieval never surfaces content beyond a user's clearance.

Developer Productivity Assistant

▪ Business Problem:

Engineers spend significant time on code review, test writing, and documentation — work that is necessary but slow.

▪ AI Solution:

An LLM-based assistant integrated into the SDLC for code review, test generation, and doc generation (Module 10 examples 3, 4, 7, 8).

▪ Architecture:

CI/CD pipeline or IDE → OCI Generative AI → review comments / generated tests / docs → pull request.

▪ OCI Services:

OCI Generative AI, API Gateway, IAM, OCI DevOps (pipeline integration).

EXAMPLE PROMPT

```
"Generate pytest unit tests for this discount-  
calculation  
function, covering normal, zero, negative, and  
rounding  
edge cases."
```

EXPECTED RESULT

A ready-to-review pytest file with parametrized test cases covering the requested edge cases.

SECURITY CONSIDERATIONS

Source code sent to the model should follow the same data-classification rules as the repository itself; avoid sending secrets/credentials in prompts.

Automated Report Generation

▪ Business Problem:

Analysts spend hours manually assembling recurring operational and financial narrative reports.

▪ AI Solution:

An LLM-based reporting layer that turns structured query results into a polished narrative summary on a schedule.

▪ Architecture:

Scheduled job → Autonomous Database query → OCI Generative AI (narrative generation) → formatted report → distribution.

▪ OCI Services:

Autonomous Database, OCI Generative AI, OCI Functions (scheduling), OCI Object Storage (report archive).

EXAMPLE PROMPT

```
"Using the attached Q3 sales-by-region query results, write a 3-paragraph executive summary highlighting the top and bottom performing regions and likely drivers."
```

EXPECTED RESULT

A concise, data-grounded executive summary generated in seconds instead of hours, with the underlying query results attached for verification.

SECURITY CONSIDERATIONS

Only pre-approved, already-authorized query results are passed to the model — the model never has direct database query access itself.

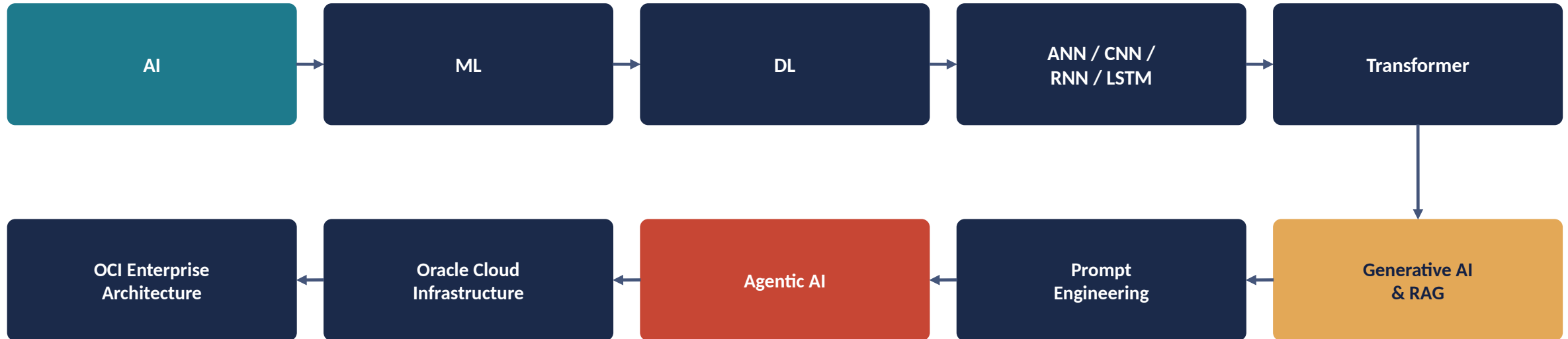
COURSE CLOSING

From Foundations to Production on OCI

Landscape recap · Adoption path · Reference architecture · Governance · Key takeaways · Hands-on exercises · Two mini-projects

The Complete AI Technology Landscape

Foundations → Architectures → Language Models → Generative & Agentic Systems → Oracle Cloud Infrastructure



Fourteen modules, one continuous technical arc — from what intelligence means to how it runs in production on OCI.

How Oracle Organizations Can Adopt AI

OBJECTIVE *A pragmatic, staged adoption path rather than an all-at-once transformation.*

- **Stage 1 — Consume:**

Start with pretrained OCI AI Services and OCI Generative AI chat/embeddings — zero training, immediate value on well-scoped problems.

- **Stage 2 — Ground:**

Add Enterprise RAG on Oracle Database 23ai Vector Search to make GenAI answers accurate and citable against your own data.

- **Stage 3 — Customize:**

Use OCI Data Science and Generative AI fine-tuning where pretrained models fall short on domain-specific accuracy or style.

- **Stage 4 — Automate:**

Introduce Agentic AI for well-governed, multi-step workflows once RAG and monitoring maturity are in place.

- Each stage should be justified by a specific business problem (Module 14 pattern) — not adopted as a technology-first initiative.

OCI AI Reference Architecture

Compute and OKE underpin all four layers wherever custom hosting is required.

Governance & Operations — IAM · Logging · Monitoring · API Gateway



Application Layer — RAG Applications · Agentic Workflows · Chat/Assistant UIs



AI / ML Layer — OCI Generative AI · OCI AI Services · OCI Data Science



Data Layer — Object Storage · Autonomous Database · Oracle DB 23ai Vector Search

Every enterprise AI system on OCI is a composition of these four layers.

Security and Governance Considerations

OBJECTIVE *The cross-cutting concerns that apply to every architecture in this course.*

- **Least-privilege access:**

Scope IAM policies and dynamic groups tightly per agent/application identity — never grant broad Generative AI or data access by default.

- **Data protection:**

Classify and, where required, redact sensitive fields before they reach a prompt, embedding, or log entry.

- **Human-in-the-loop for high-impact actions:**

Agentic AI's automated tool calls should require human approval for irreversible or high-risk actions (Module 11).

- **Auditability:**

Log every prompt, retrieval, and tool call via OCI Logging for traceability and compliance review.

- **Output validation:**

- Treat every GenAI output as a draft requiring validation proportional to its business impact — never as ground truth by default.

Key Takeaways

OBJECTIVE *The load-bearing ideas from all 14 modules.*

- AI, ML, DL, GenAI, and Agentic AI are nested, progressively specialized technologies — not synonyms; choose the least complex one that solves the actual business problem.
- Every deep architecture (ANN, CNN, RNN, LSTM, Transformer) is built from the same primitives: weights, bias, activation, forward pass, loss, and backpropagation.
- The Transformer's self-attention mechanism is the single architectural idea underlying every modern LLM, and therefore every GenAI and Agentic AI system.
- RAG and fine-tuning solve different problems — grounding in current facts versus changing behavior/style — and are frequently used together.
- Agentic AI is GenAI plus planning, tools, and memory, wrapped in guardrails; autonomy should be earned incrementally, not granted by default.
- OCI provides a complete, layered AI stack — from pretrained APIs to full custom ML — with Oracle Database 23ai Vector Search as a distinctive advantage: retrieval co-located with enterprise data.

Hands-On Exercises

OBJECTIVE *Practical exercises reinforcing each part of the curriculum.*

- **Exercise 1 (Modules 1–2):**

Classify 10 real workplace AI examples as Narrow AI, rule-based, or ML-based, and justify each.

- **Exercise 2 (Module 2):**

Train and evaluate a scikit-learn classifier on a provided tabular dataset; report precision/recall and diagnose over/underfitting.

- **Exercise 3 (Modules 3–8):**

Build and train a small CNN and a small LSTM on provided datasets; compare their architectures and training curves.

- **Exercise 4 (Module 8):**

Hand-compute self-attention weights for a 4-token toy sentence given sample Q/K vectors.

- **Exercise 5 (Module 10):**

Rewrite 5 of your own team's real prompts using the Role/Context/Task/Constraints/Examples/Format framework.

- **Exercise 6 (Module 12):**

- Map an AI idea from your own team to the correct OCI service layer (AI Services vs. Generative AI vs. Data Science).

Build an OCI-Based Enterprise RAG Assistant

OBJECTIVE *Apply Modules 9, 10, 12, and 13 end to end.*

- **Objective:**

Build a working RAG assistant answering questions from a provided set of Oracle documentation or company policy PDFs.

- **Steps:**

1) Chunk and embed the source documents. 2) Store embeddings in Oracle Database 23ai using AI Vector Search. 3) Build a retrieval + prompt-assembly layer. 4) Call OCI Generative AI for grounded generation. 5) Add citation of source passages.

- **Deliverable:**

A working chat interface (CLI or simple web app) that answers questions with cited, grounded responses and gracefully declines when the answer isn't in the corpus.

- **Services used:**

OCI Generative AI, Oracle Database 23ai Vector Search, Object Storage, API Gateway.

Build an Agentic AI Assistant Using OCI Tools

OBJECTIVE *Apply Modules 10, 11, 12, and 13 end to end.*

- **Objective:**

Build an agent that resolves a realistic multi-step task using at least two distinct tools.

- **Suggested scenario:**

An IT-operations agent that reads a simulated alert, queries a sample database for recent changes, reasons about the likely cause, and drafts (but does not auto-execute) a remediation recommendation.

- **Steps:**

1) Define your tool schemas (function name, parameters, description). 2) Implement the ReAct loop (Thought → Action → Observation). 3) Add a human-approval gate before any state-changing action. 4) Log every step for auditability.

- **Services used:**

OCI Generative AI Agents (or a custom Function-Calling loop), OCI Functions, IAM, Logging.

Thank You

AI, Machine Learning, Deep Learning, Generative AI & Agentic AI with Oracle Cloud Infrastructure